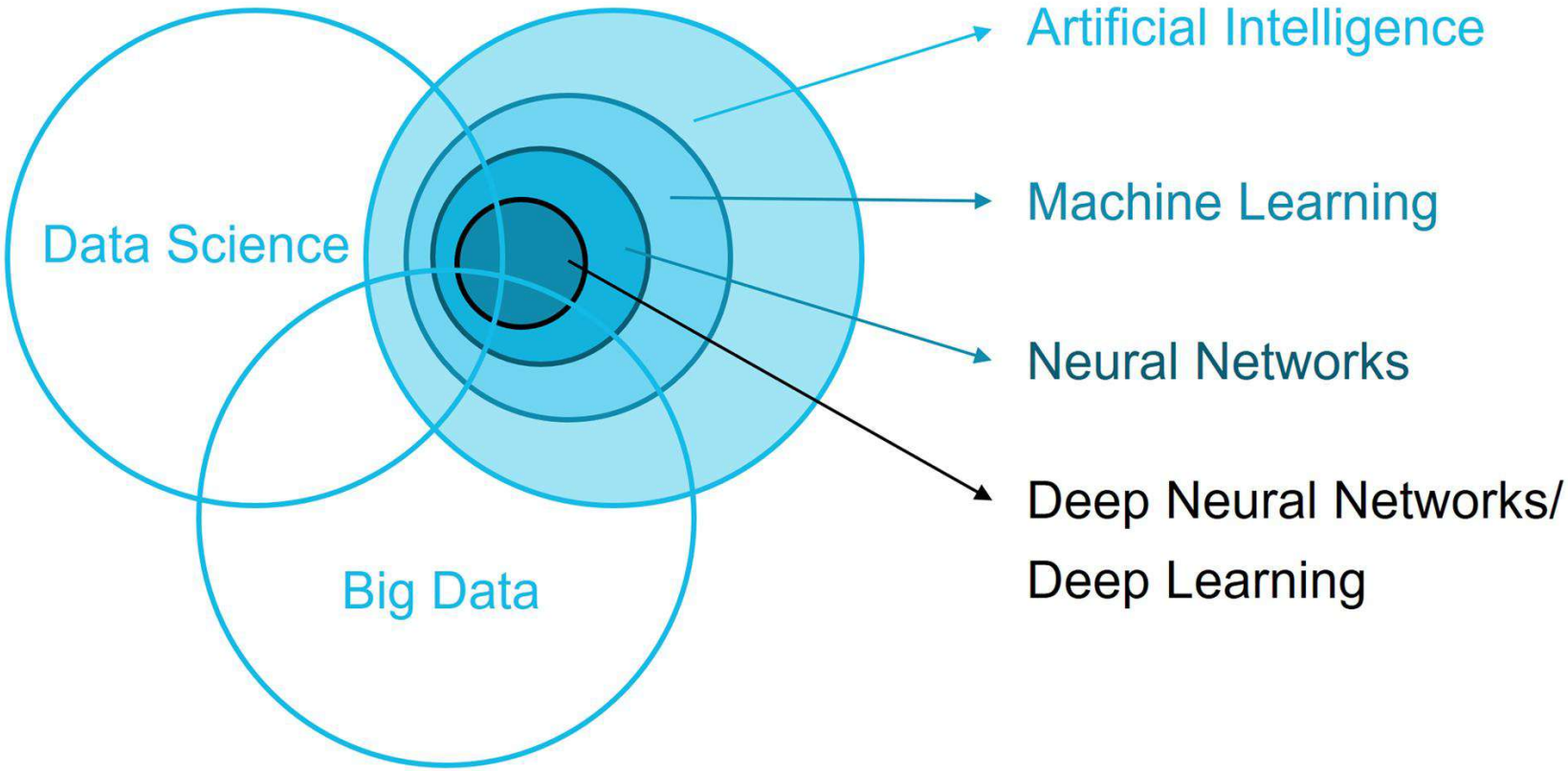
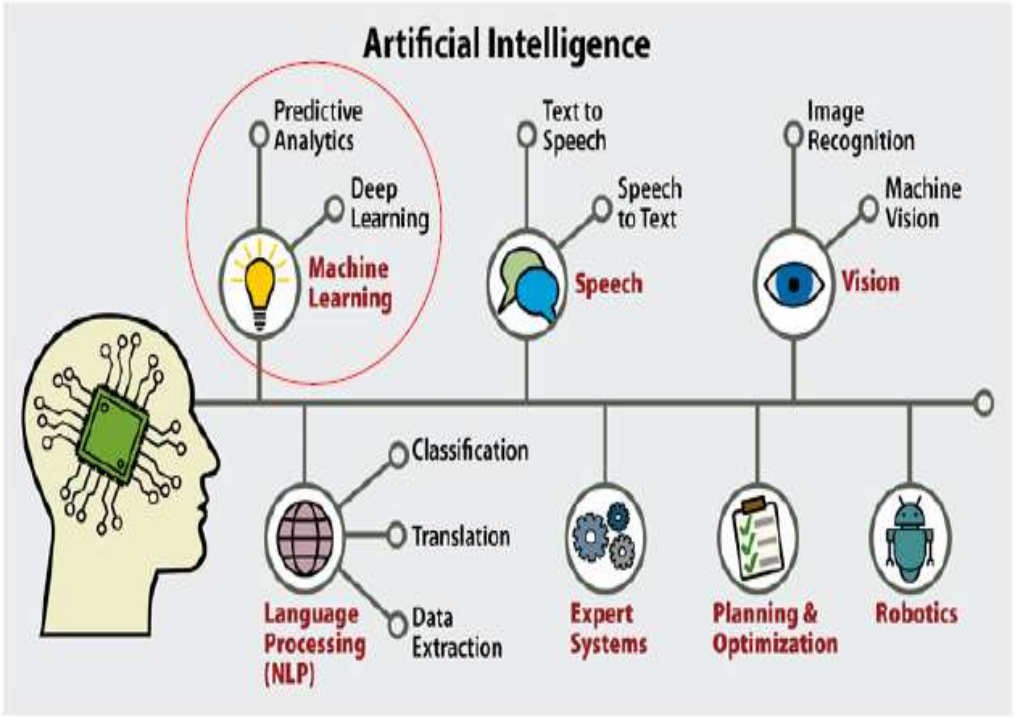
Deep Learning

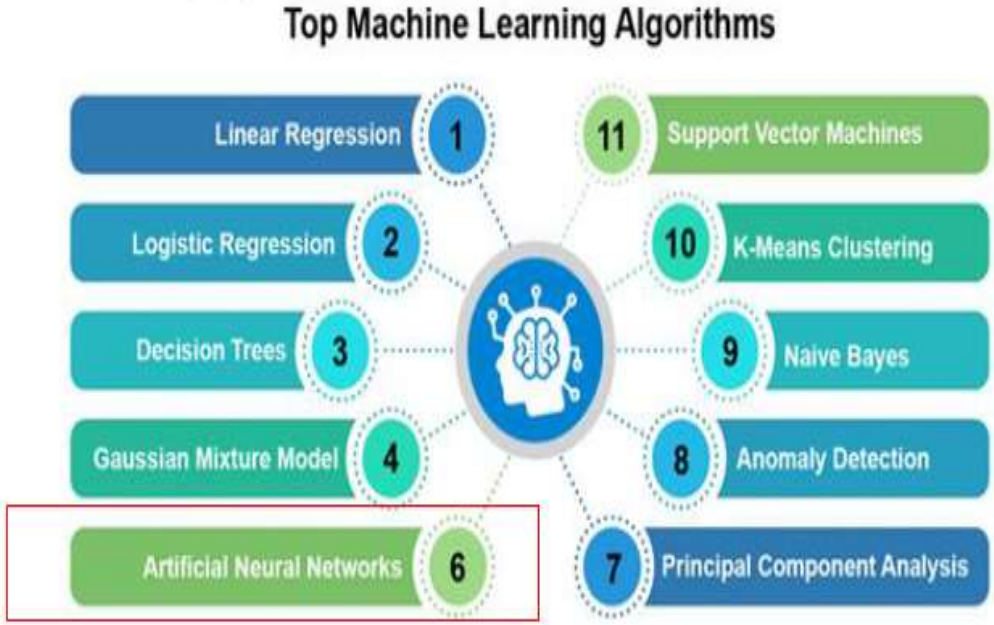
Module 1

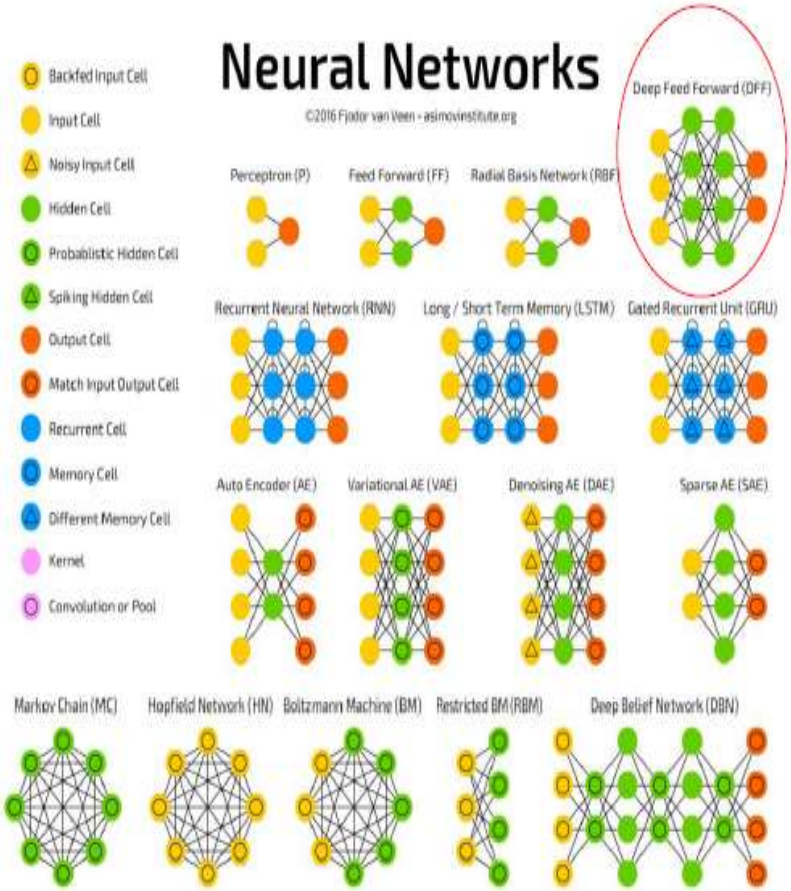
Syllabus-Module 1

- Review of Neural Networks: Model of a biological neuron, McCulloch Pitts Neuron, Activation Functions, Perceptron, Perceptron Learning Algorithm and Convergence, Multilayer Perceptron, Back propagation, Learning XOR, Sigmoid Neurons, Gradient Descent, Feed forward Neural Networks.


Data Science
Big Data
Artificial Intelligence
Machine Learning
Neural Networks
Deep Neural Networks/
Deep Learning


Artificial Intelligence
Predictive Analytics
Deep Learning
Machine Learning
Text to Speech
Speech to Text
Speech
Image Recognition
Machine Vision
Vision
Classification
Translation
Data Extraction
Language Processing (NLP)
Expert Systems
Planning & Optimization
Robotics

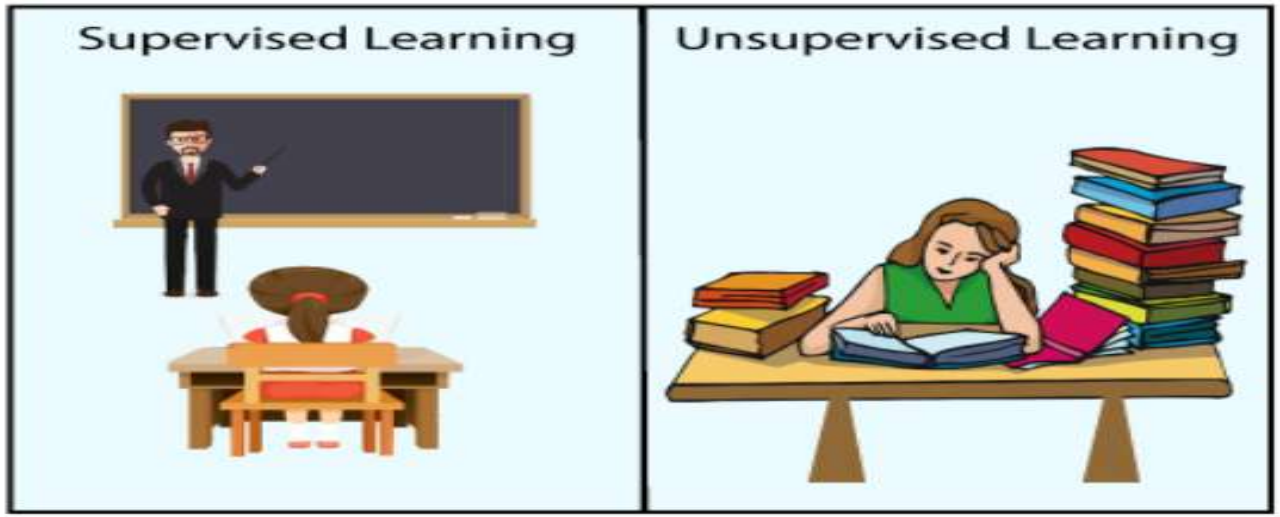

Top Machine Learning Algorithms
1 Linear Regression
2 Logistic Regression
3 Decision Trees
4 Gaussian Mixture Model
6 Artificial Neural Networks
11 Support Vector Machines
10 K-Means Clustering
9 Naive Bayes
8 Anomaly Detection
7 Principal Component Analysis


Neural Networks
©2016 Fjodor van Veen - asimovinstitute.org
Backfed Input Cell
Input Cell
Noisy Input Cell
Hidden Cell
Probabilistic Hidden Cell
Spiking Hidden Cell
Output Cell
Match Input Output Cell
Recurrent Cell
Memory Cell
Different Memory Cell
Kernel
Convolution or Pool
Perceptron (P)
Feed Forward (FF)
Radial Basis Network (RBF)
Deep Feed Forward (DFF)
Recurrent Neural Network (RNN)
Long / Short Term Memory (LSTM)
Gated Recurrent Unit (GRU)
Auto Encoder (AE)
Variational AE (VAE)
Denoising AE (DAE)
Sparse AE (SAE)
Markov Chain (MC)
Hopfield Network (HN)
Boltzmann Machine (BM)
Restricted BM (RBM)
Deep Belief Network (DBN)

AI and Machine Learning

- Artificial intelligence is a field of computer science which makes a computer system that can mimic human intelligence. It is comprised of two words "**Artificial**" and "**intelligence**", which means "a human-made thinking power."
- *Machine learning is a subfield of artificial intelligence, which enables machines to learn from past data or experiences without being explicitly programmed.*

Supervised Vs UnSupervised learning



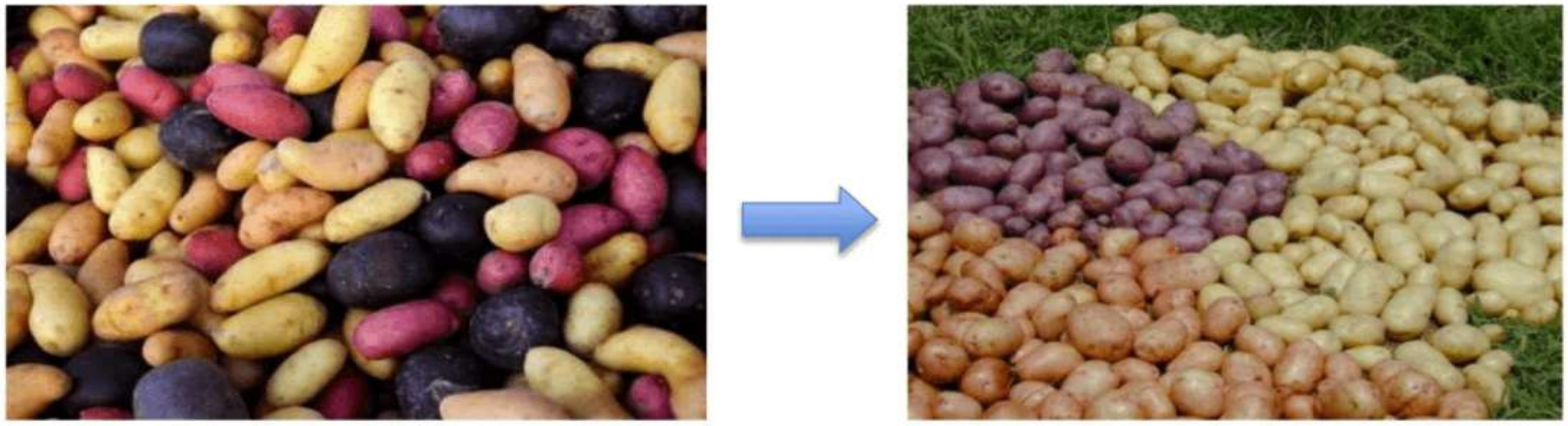
Supervised learning is a machine learning method in which models are trained using labeled data. In supervised learning, models need to find the mapping function to map the input variable (X) with the output variable (Y).

Y=f(X)

Supervised learning can be used for two types of problems: **Classification** and **Regression**.

UnSupervised Learning

- Unsupervised learning is another machine learning method in which patterns inferred from the unlabeled input data, or dividing the data into homogeneous partitions
- Unsupervised learning can be used for **Clustering** and **Association**.



sample

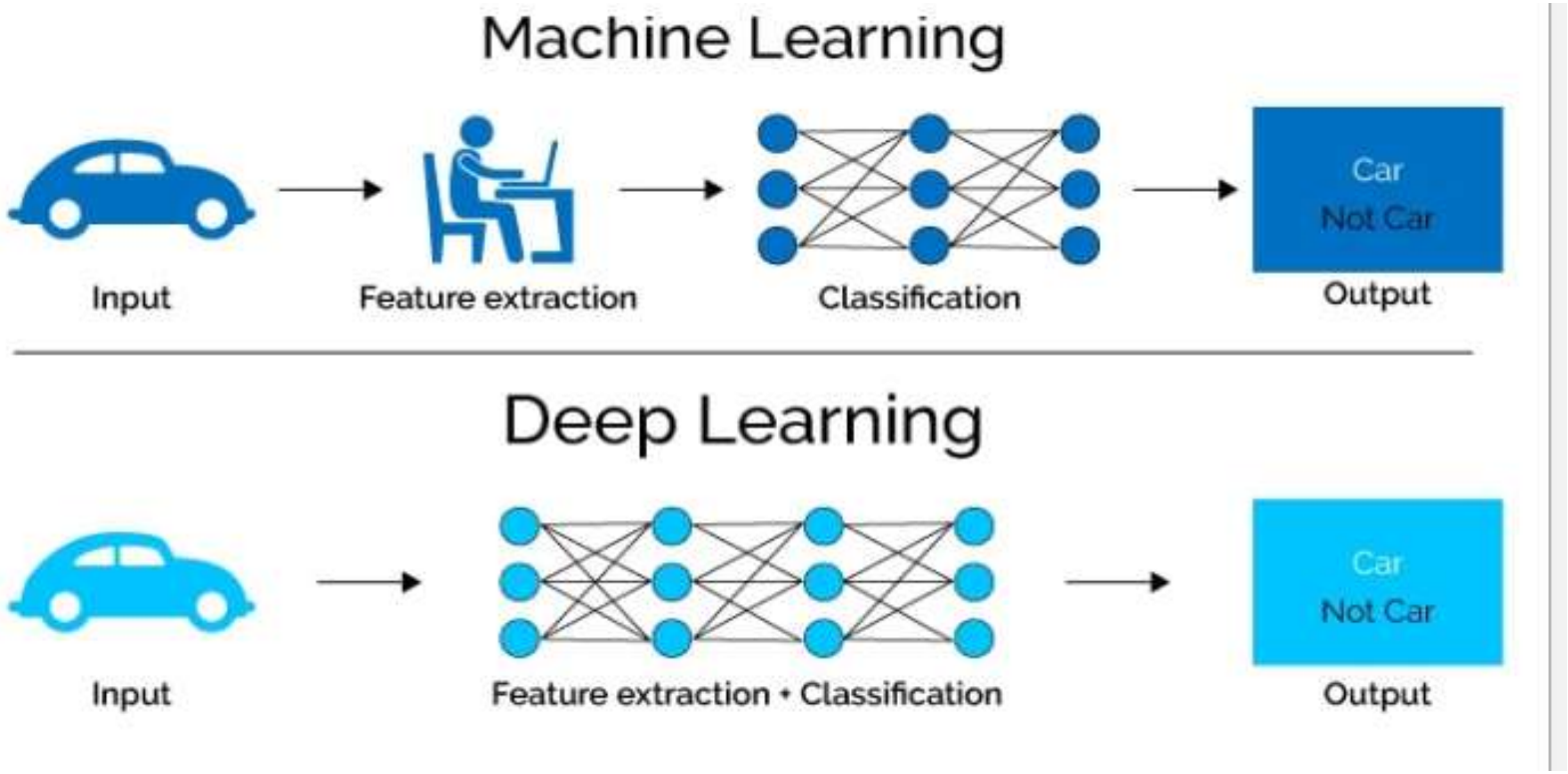
Cluster/group

Reinforcement Learning

- It is the problem of getting an agent to act in the world so as to maximize its rewards.

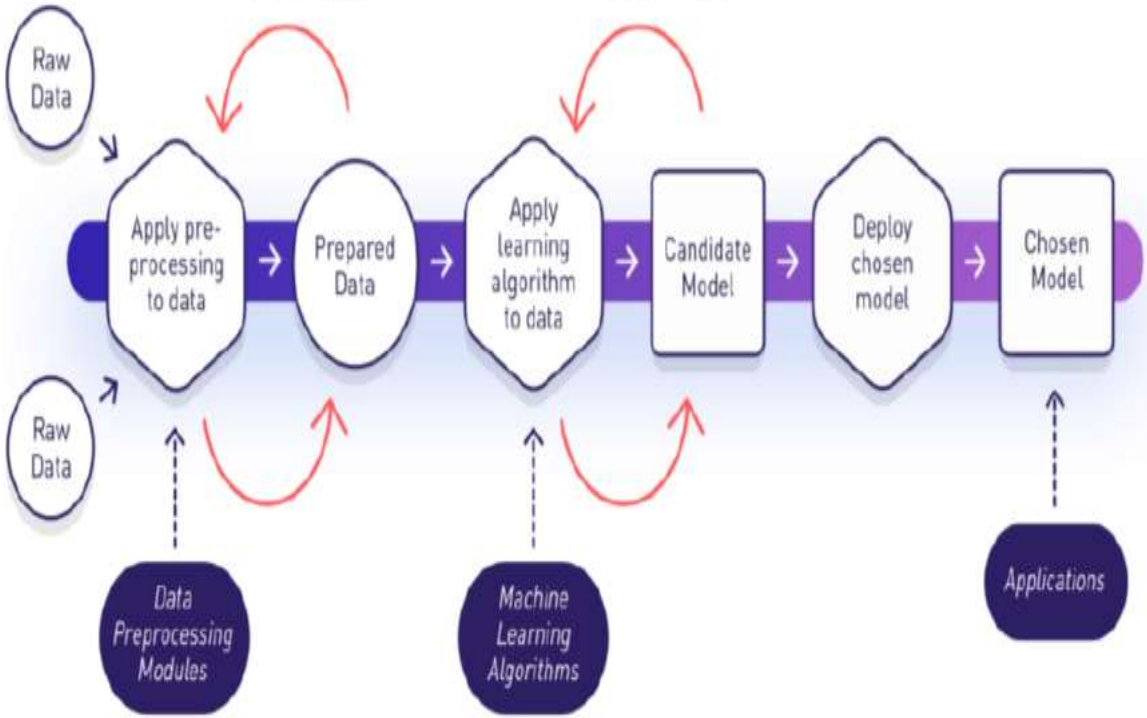


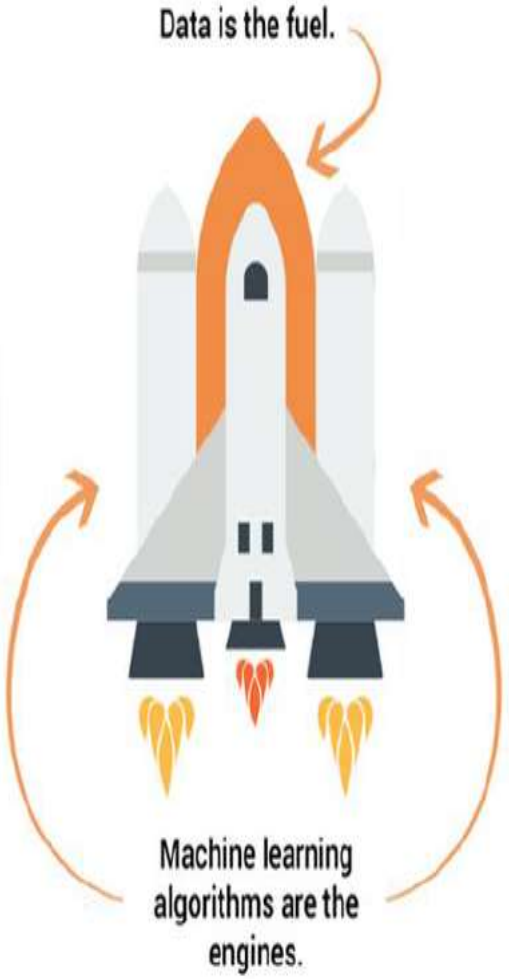
Machine Learning Vs Deep Learning



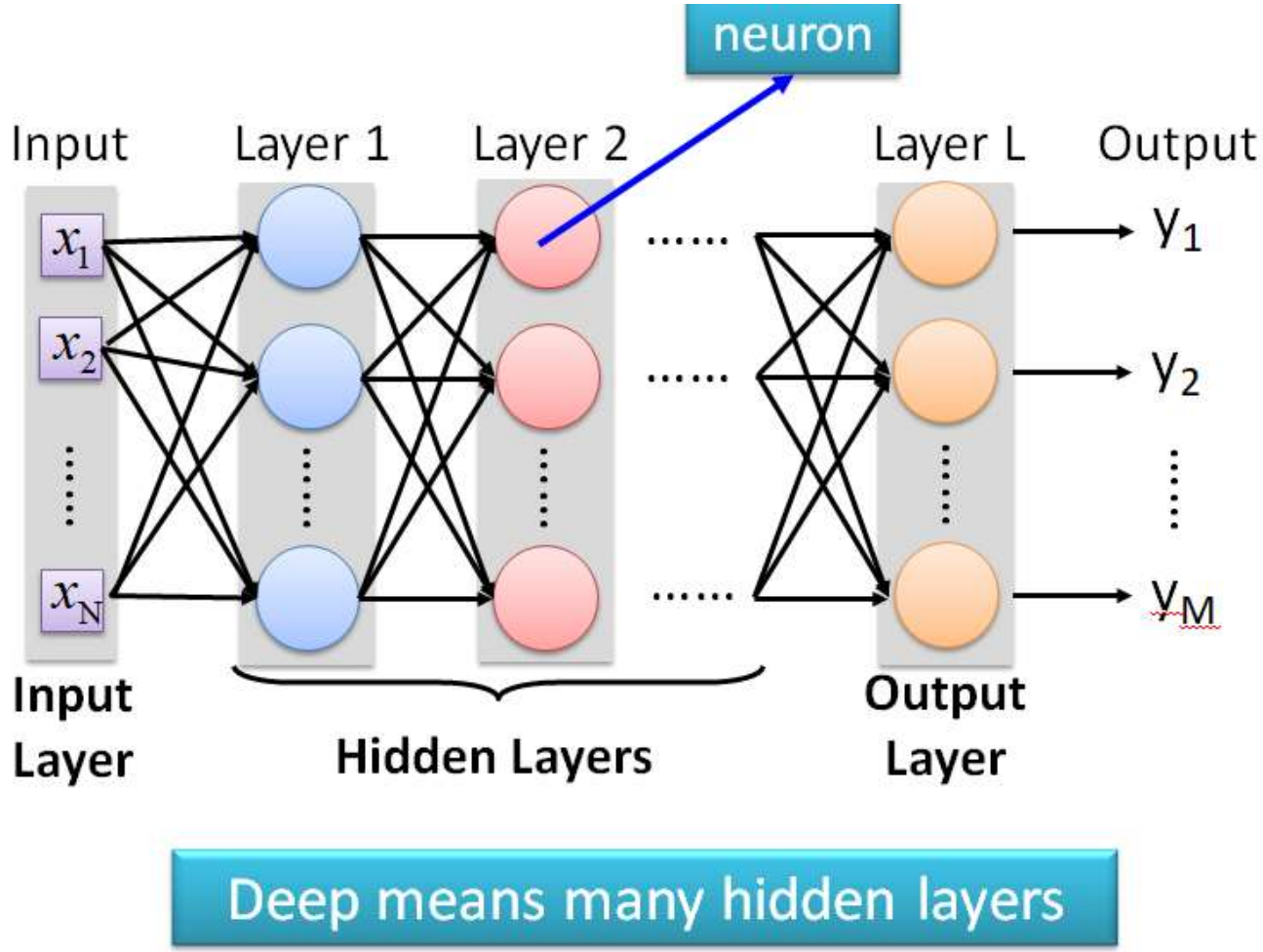
Deep Learning: A technique to perform machine learning inspired by our own network of neurons

Machine Learning Pipeline





Deep Learning



Deep means many hidden layers

Application Areas of Deep Learning

- Language modelling,
- Question answering
- Computer vision
- Object detection
- Visual tracking (track the same person in a video)
- Image/ Video captioning (generate captions for images)
- Visual question answering (Given an image/ Video & a question, answer that question.

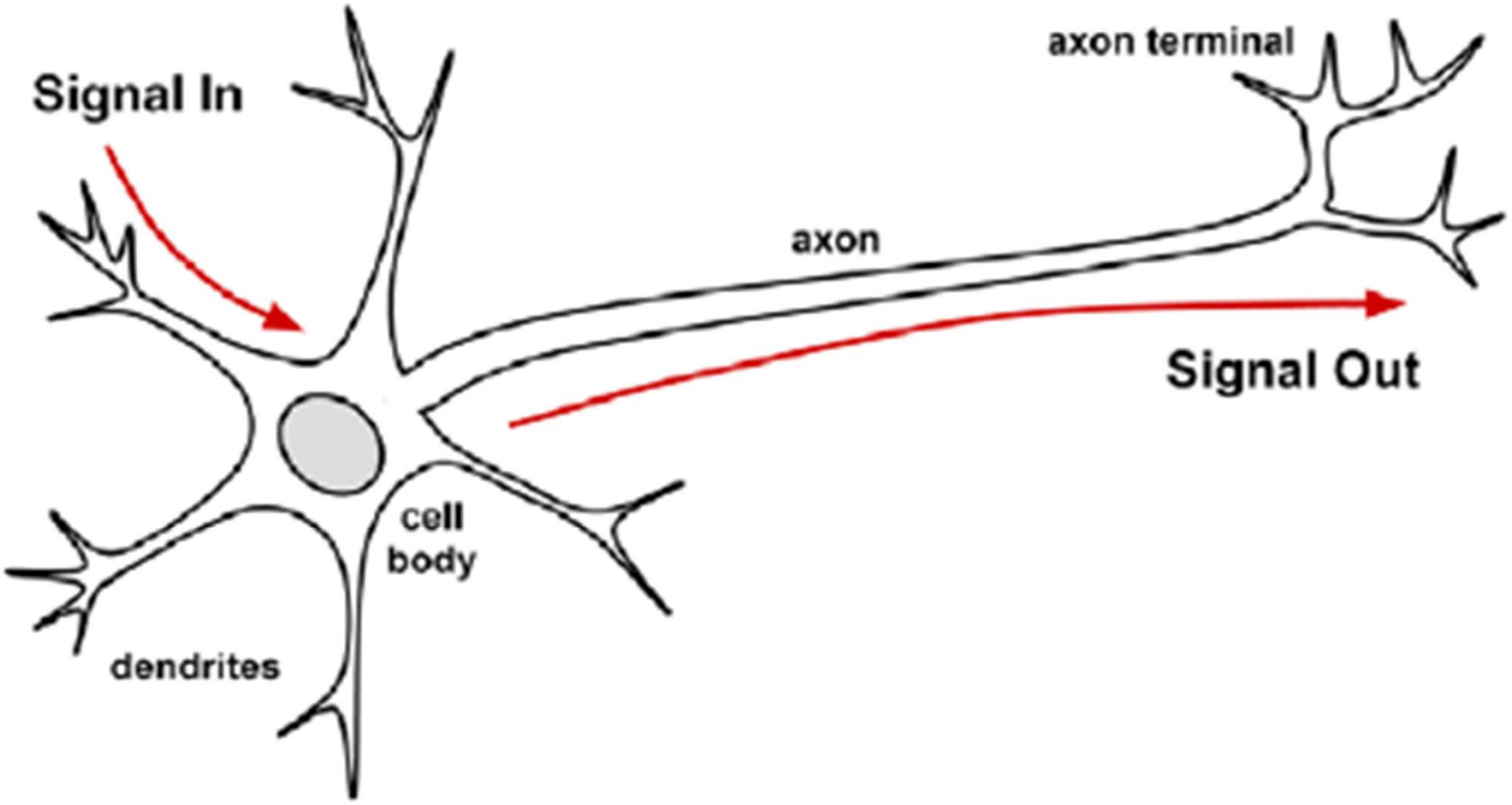
Applications of Deep Learning

- Video summarizations; if you are given a large video and you want to generate a trailer, a sort of a trailer for that video contains, which kind is the most important frame for that video. Even these systems are based on deep learning.
- Classification
- Recognition
- Image/ Video/ Music Generation (Generative Adversarial Networks, Variational Autoencoders)

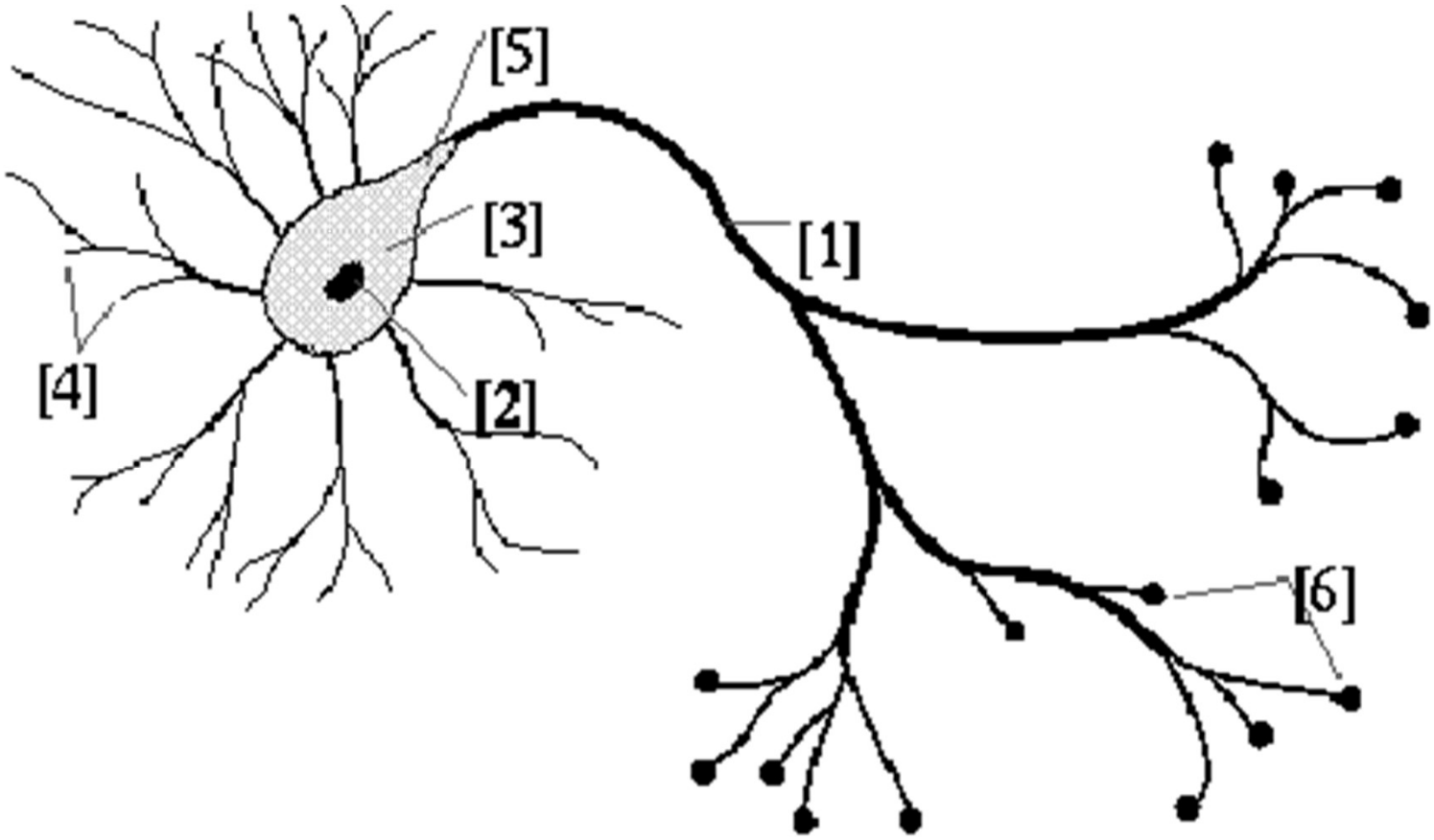
Neural Networks

- Neural networks use concepts borrowed from Animal brains in order to model arbitrary functions
- An **Artificial Neural Network (ANN)** models the relationship between a set of input signals and an output signal using a model derived from our understanding of how a biological brain responds to stimuli from sensory inputs.
- Brain uses a network of interconnected cells called **neurons** to create a massive parallel processor

Biological Motivation



Biological Neuron



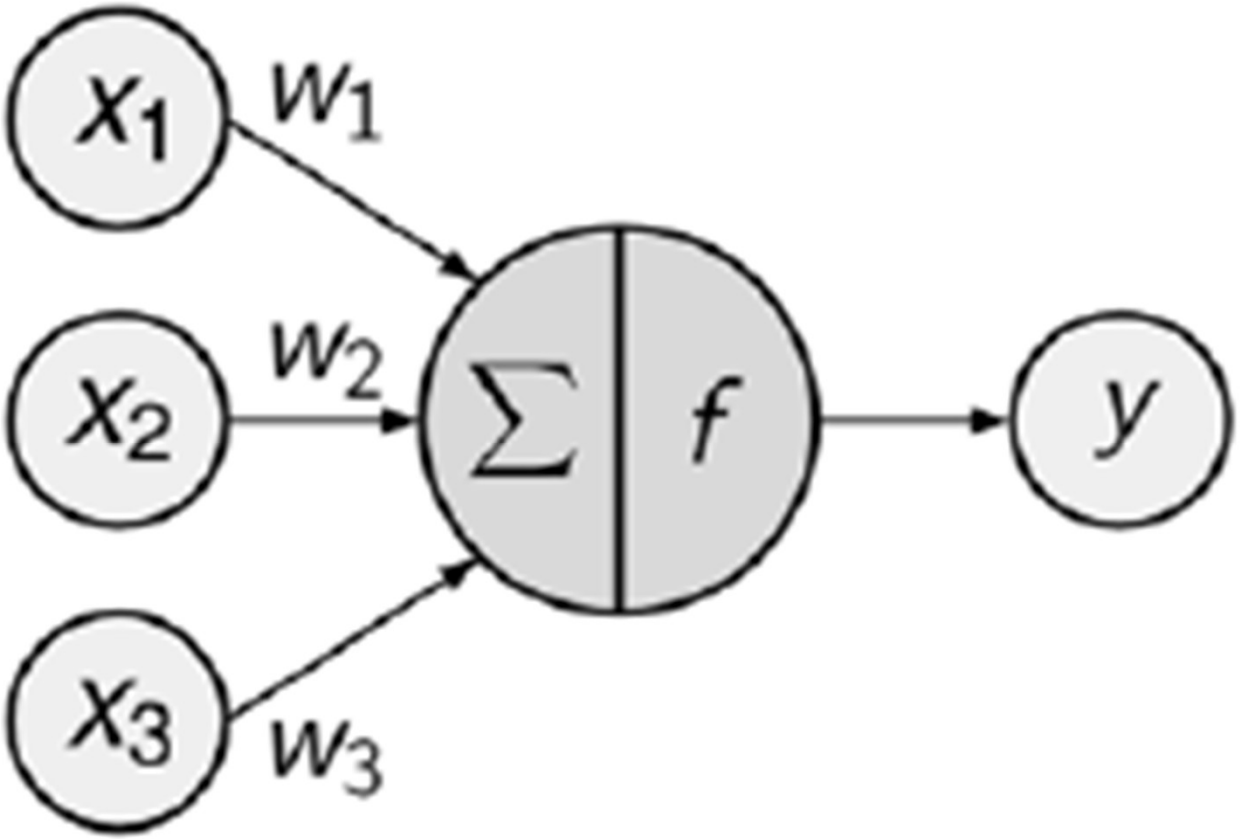
1.Axon 2. Nucleus 3.Soma (Body) 4. Dendrite 5. Axon Hillock 6. Terminals (Synapses)

Biological Neuron

- The fundamental element of the neural network is called a neuron.
- a neuron mainly consists of three parts:
  - Dendrites, Soma, and Axon.
- Dentrites are the tree-like structure that receives the signal from surrounding neurons, where each line is connected to one neuron.
- Axon is a thin cylinder that transmits the signal from one neuron to others.
- At the end of axon, the contact to the dendrites is made through a synapse.
- A neuron fires an electrical impulse only if certain condition is met

- incoming signals are received by the cell‘s **dendrites** through a biochemical process.
- that allows the impulse to be weighted according to its relative importance.
- As the cell body begins to accumulate the incoming signals, a threshold is reached at which the cell fires
- The output signal is then transmitted via an electrochemical process through the **axon.**
- At the axon's terminals, the electric signal is again processed as a biochemical signal
- And passed to the neighboring neurons across a tiny gap known as a **synapse.**

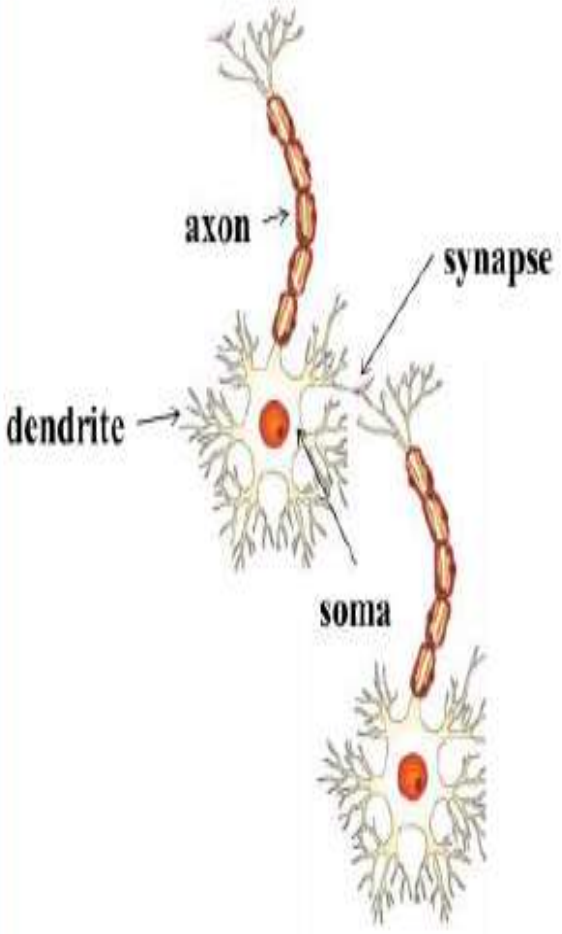
A single artificial neuron (Perceptron)



- The input signals received by the dendrites(*x variables) and the output signal (y variable).*
- each dendrite's signal is weighted (*w values) according to its importance*
- The input signals are summed by the cell body

and the signal is passed on to an **activation function** denoted by ***f.***

Biological Neuron



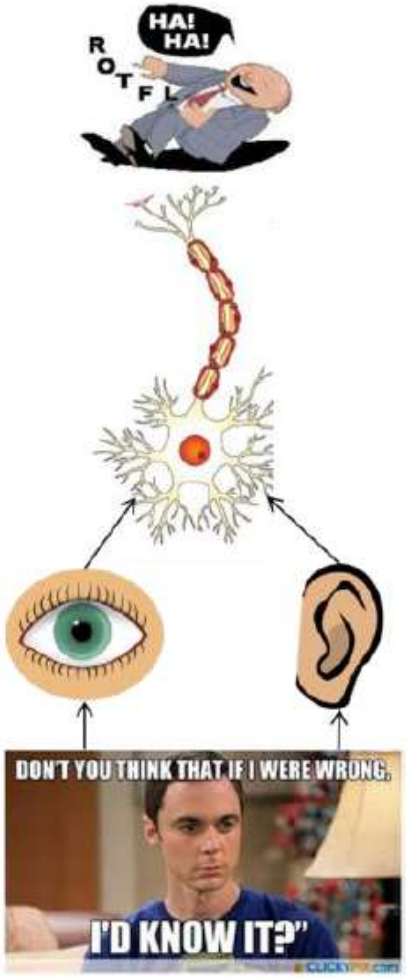
Biological Neurons*

dendrite: receives signals from other neurons

synapse: point of connection to other neurons

soma: processes the information

axon: transmits the output of this neuron

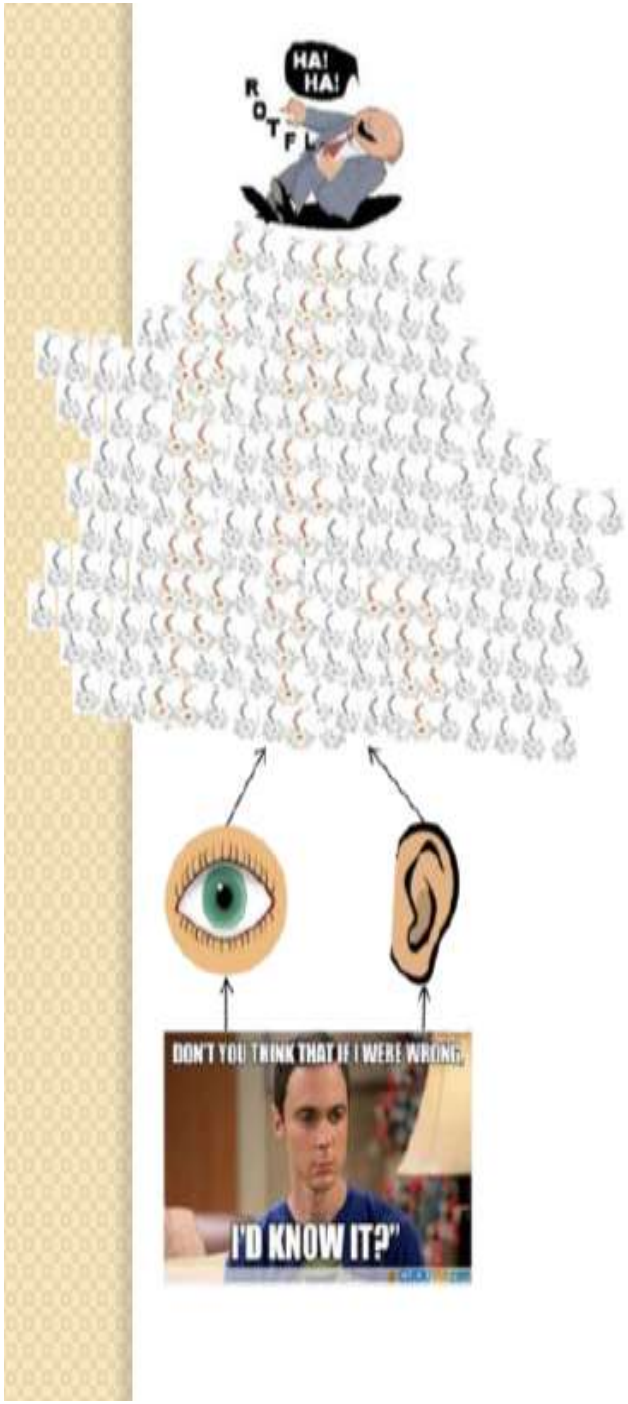


Let us see a cartoonish illustration of how a neuron works

Our sense organs interact with the outside world

They relay information to the neurons

The neurons (may) get activated and produces a response (laughter in this case)



Of course, in reality, it is not just a single neuron which does all this

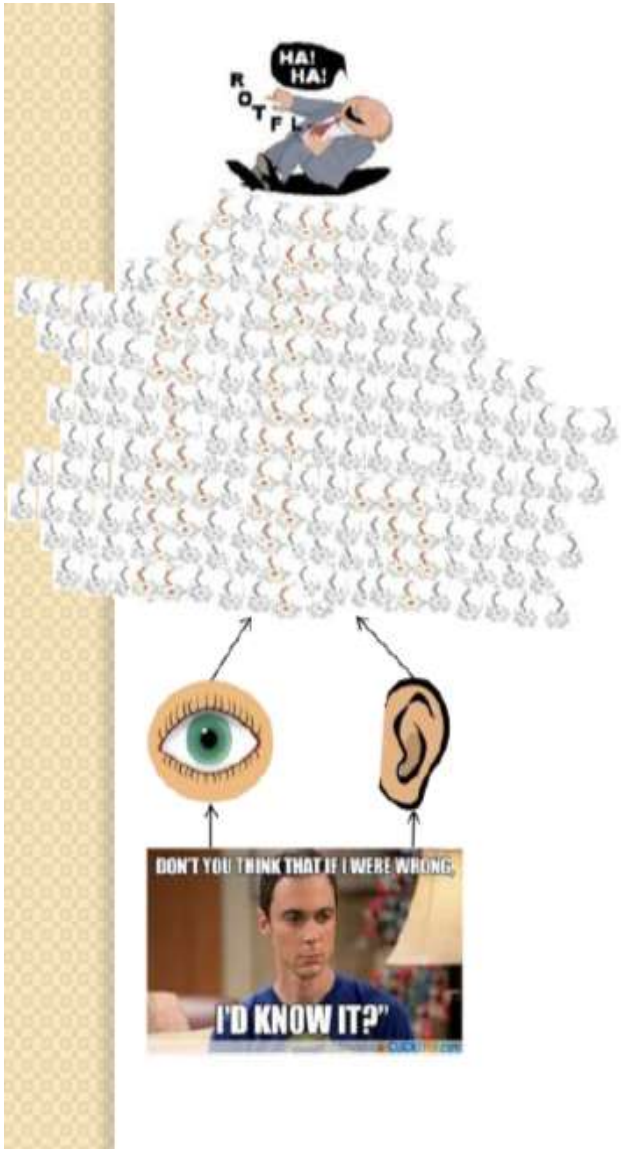
There is a massively parallel interconnected network of neurons

The sense organs relay information to the lowest layer of neurons

Some of these neurons may fire (in red) in response to this information and in turn relay information to other neurons they are connected to

These neurons may also fire (again, in red) and the process continues eventually resulting in a response (laughter in this case)

An average human brain has around 10^{11} (100 billion) neurons!



This massively parallel network also ensures that there is division of work

Each neuron may perform a certain role or respond to a certain stimulus



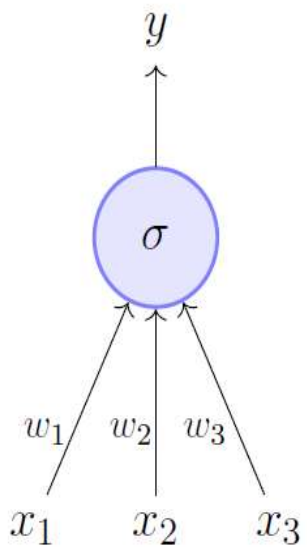


Asimplified illustration

This massively parallel network also ensures that there is division of work

Each neuron may perform a certain role or respond to a certain stimulus

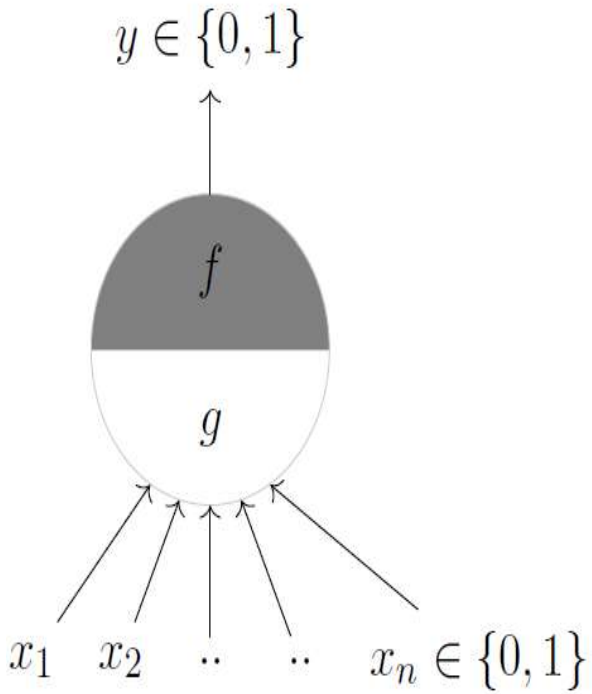
Artificial Neuron



Artificial Neuron

- The most fundamental unit of a deep neural network is called an *artificial neuron*
- Why is it called a neuron ? Where does the inspiration come from ?
- The inspiration comes from biology (more specifically, from the *brain*)
- *biological neurons* = *neural cells* = *neural processing units*
- We will first see what a biological neuron looks like ...

McCulloch Pitts Neuron



- McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)
- g aggregates the inputs and the function f takes a decision based on this aggregation
- The inputs can be excitatory or inhibitory
- $y = 0$ if any x_i is inhibitory, else

$$g(x_1, x_2, ..., x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

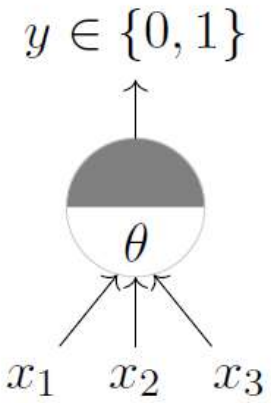
$$y = f(g(\mathbf{x})) = 1 \quad if \quad g(\mathbf{x}) \geq \theta$$
$$= 0 \quad if \quad g(\mathbf{x}) < \theta$$

- θ is called the thresholding parameter
- This is called Thresholding Logic

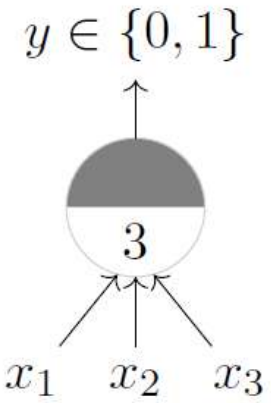
Excitatory and Inhibitory in neural networks

- Neurons communicate through electrical currents called action potentials, which are either excitatory or inhibitory.
- Excitatory currents are those that prompt one neuron to share information with the next through an action potential.
- Inhibitory currents reduce the probability that such a transfer will take place.
- In artificial neural networks, excitatory and inhibitory inputs are typically represented using mathematical weights.
- Excitatory weights are positive, meaning they amplify the input signal, while inhibitory weights are negative, meaning they reduce the input signal's impact.

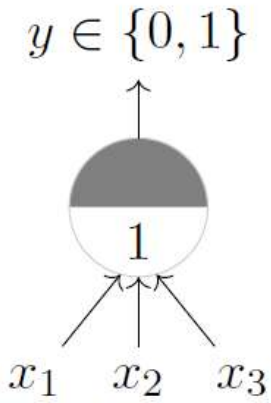
McCulloch Pitts Neuron



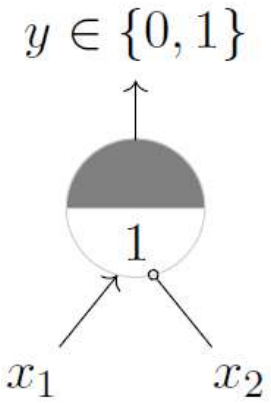
A McCulloch Pitts unit



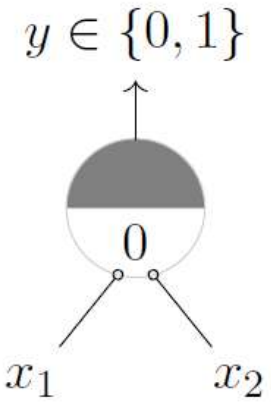
AND function



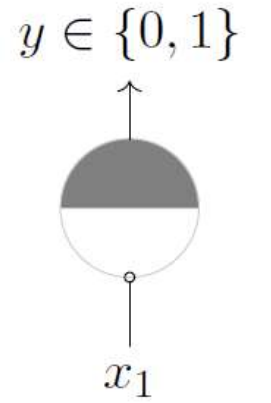
OR function



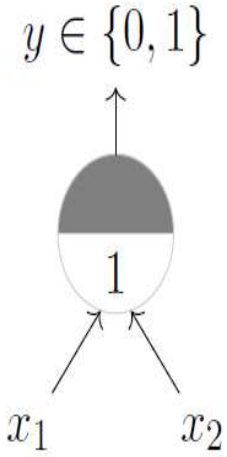
x_1 AND $!x_2$*



NOR function

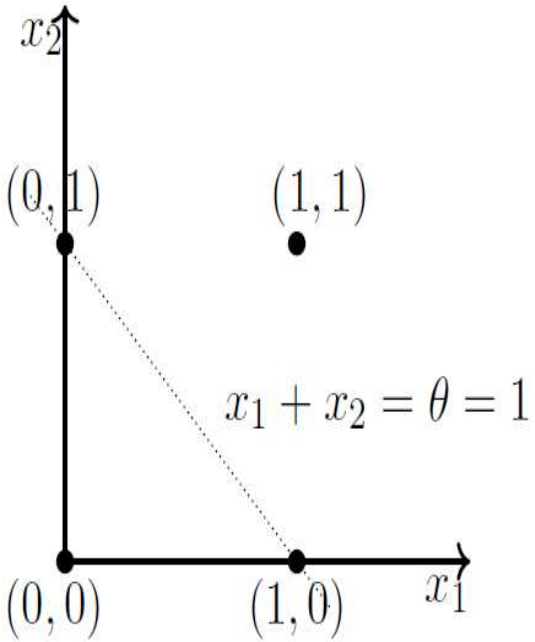


NOT function

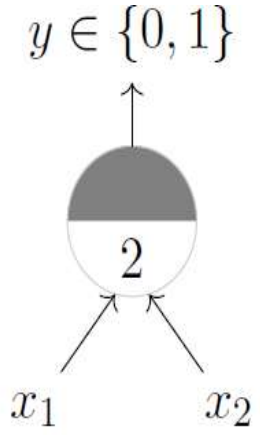


OR function

$x_1 + x_2 = \sum_{i=1}^{2} x_i \geq 1$

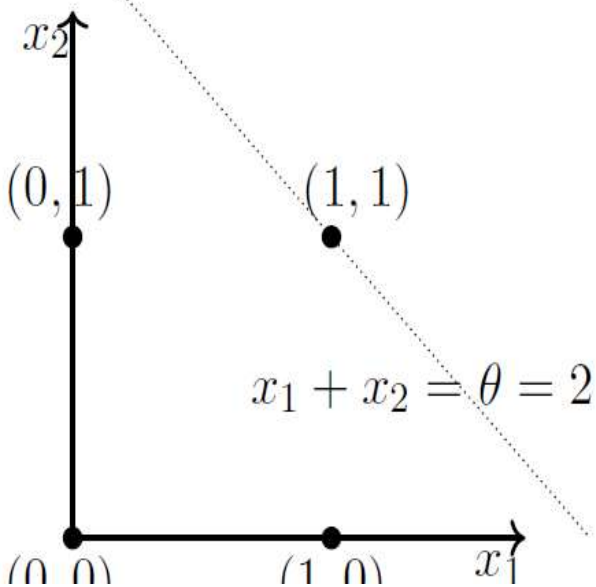


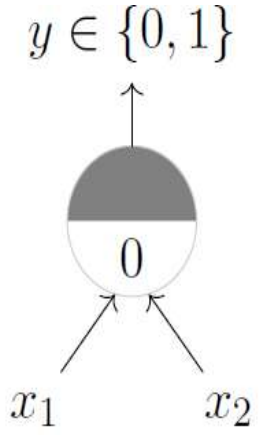
- A single MP neuron splits the input points (4 points for 2 binary inputs) into two halves
- Points lying on or above the line $\sum_{i=1}^{n} x_i - \theta = 0$ and points lying below this line
- In other words, all inputs which produce an output 0 will be on one side ($\sum_{i=1}^{n} x_i < \theta$) of the line and all inputs which produce an output 1 will lie on the other side ($\sum_{i=1}^{n} x_i \geq \theta$) of this line
- Let us convince ourselves about this with a few more examples (if it is not already clear from the math)



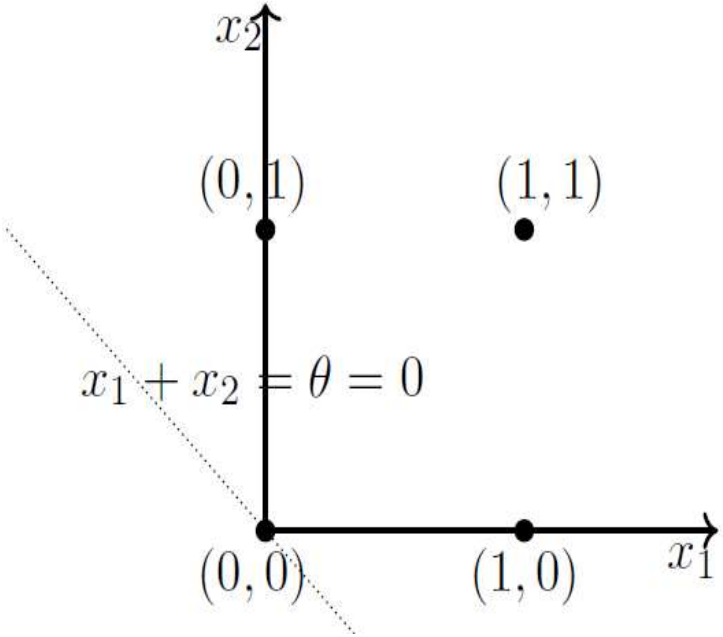
AND function

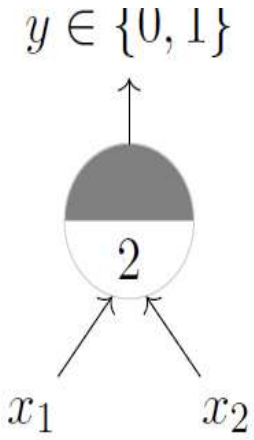
$x_1 + x_2 = \sum_{i=1}^{2} x_i \geq 2$





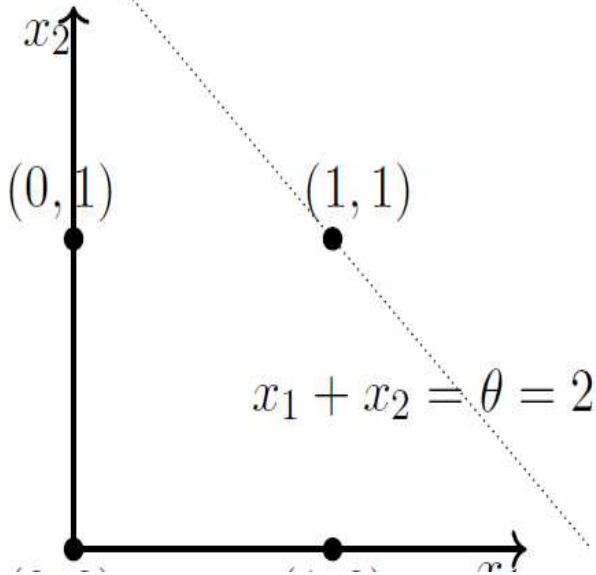
Tautology (always ON)

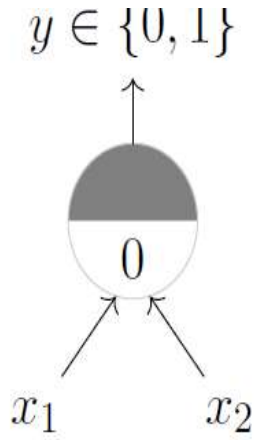




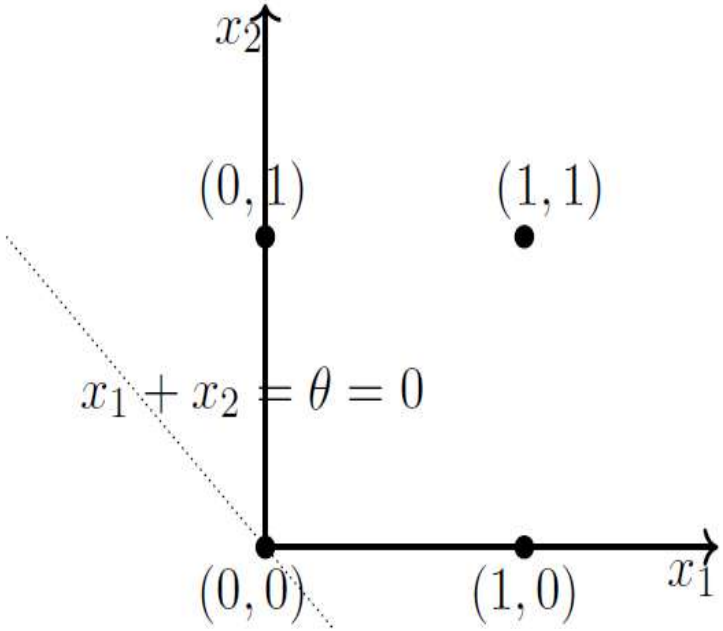
AND function

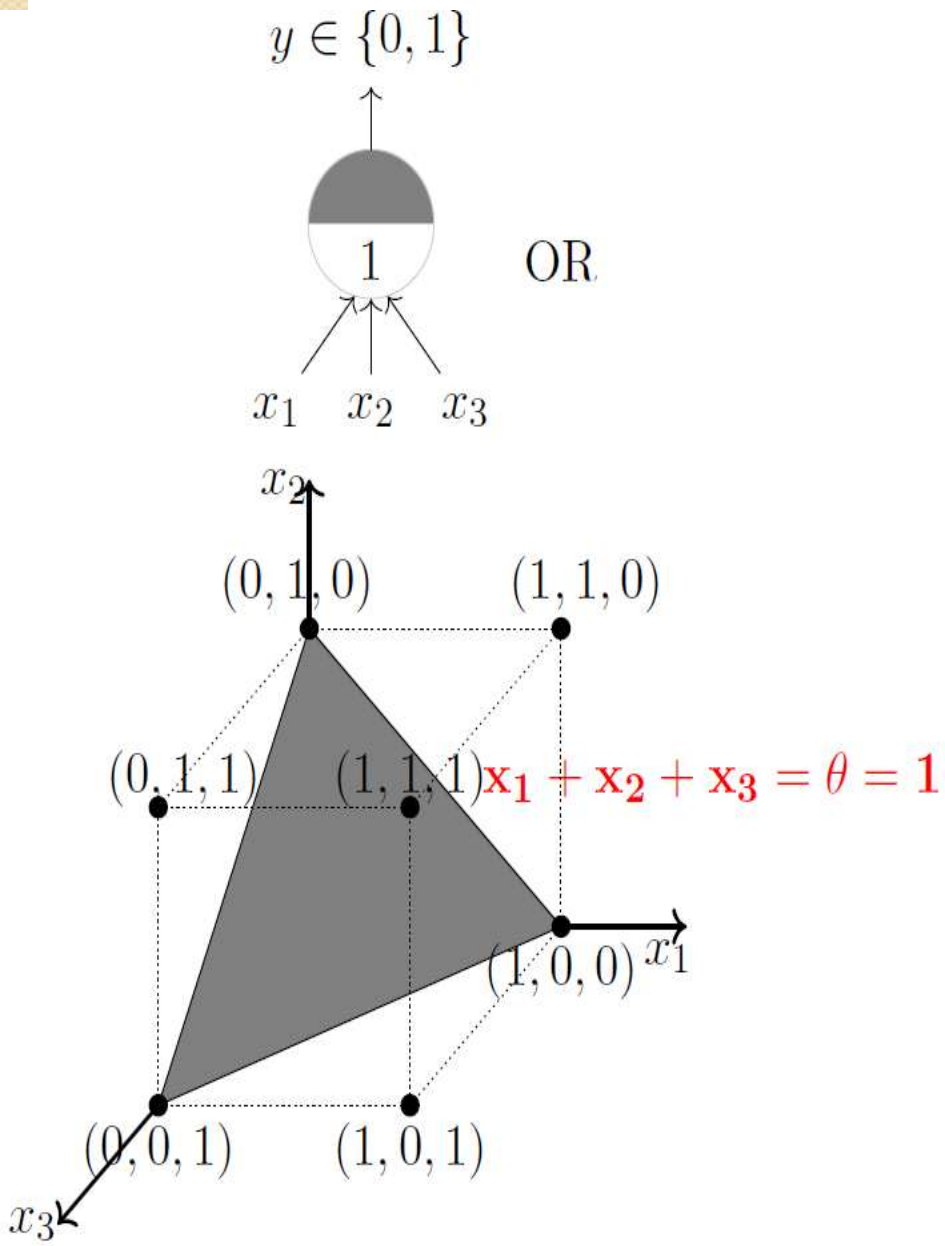
$x_1 + x_2 = \sum_{i=1}^{2} x_i \geq 2$





Tautology (always ON)





- What if we have more than 2 inputs?
- Well, instead of a line we will have a plane
- For the OR function, we want a plane such that the point (0,0,0) lies on one side and the remaining 7 points lie on the other side of the plane

Boolean gates-using MP Neuron

- **Excitatory input** — contributes +1 to the sum
- **Inhibitory input** — if it's active (1), it **blocks firing completely**, no matter what the threshold or excitatory sum is (absolute inhibition)

Rule:

$$y = 1 \text{ if (no inhibitory input is active) AND (sum of excitatory inputs} \geq \theta)$$

1. NOT Gate

Treat the single input as **inhibitory**, with no excitatory input and $\theta = 0$.

- $x = 0 \rightarrow$ not inhibited $\rightarrow$ sum $= 0 \geqslant 0 \rightarrow y = 1$
- $x = 1 \rightarrow$ inhibited $\rightarrow y = 0$ (forced)

x	y
0	1
1	0

Boolean gates-using MP Neuron

2. NOR Gate

Both x1 and x2 are **inhibitory**. $\theta = 0$ (no excitatory inputs needed).

- If either input is 1 → inhibited → y = 0
- If both are 0 → not inhibited → sum = 0 ⩾ 0 → y = 1

x1	x2	y
0	0	1
0	1	0
1	0	0
1	1	0

Boolean gates-using MP Neuron

Workaround: build NAND as AND → NOT (2 layers, still weight-free in the MP sense):

Layer 1 — AND neuron: x1, x2 excitatory, θ = 2 (fires only when both are 1)

x1	x2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Layer 2 — NOT neuron: takes AND's output as inhibitory input, θ = 0

AND	NAND
0	1
1	0

Boolean gates-using MP Neuron

Combined:

x1	x2	NAND
0	0	1
0	1	1
1	0	1
1	1	0

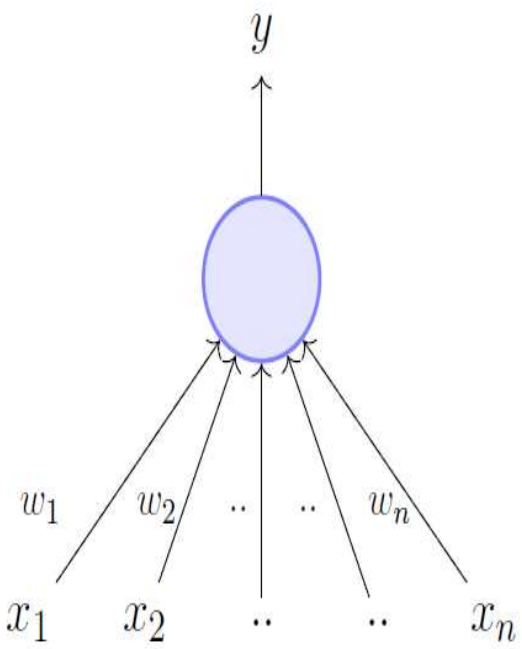
The story so far ...

- A single McCulloch Pitts Neuron can be used to represent boolean functions which are linearly separable
- Linear separability (for boolean functions) : There exists a line (plane) such that all inputs which produce a 1 lie on one side of the line (plane) and all inputs which produce a 0 lie on other side of the line (plane)

The story ahead ...

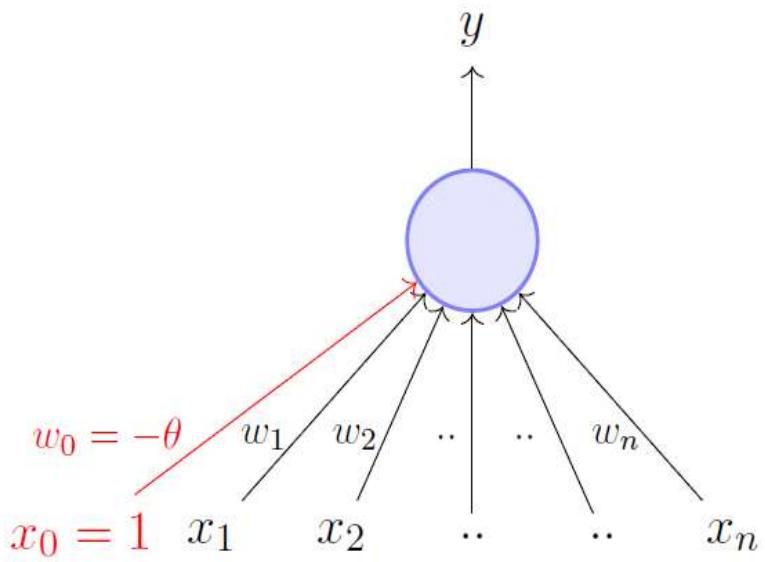
- What about non-boolean (say, real) inputs ?
- Do we always need to hand code the threshold ?
- Are all inputs equal ? What if we want to assign more weight (importance) to some inputs ?
- What about functions which are not linearly separable ?

Perceptron



- Frank Rosenblatt, an American psychologist, proposed the **classical perceptron** model (1958)
- A more general computational model than McCulloch–Pitts neurons
- **Main differences:** Introduction of numerical weights for inputs and a mechanism for learning these weights
- Inputs are no longer limited to boolean values
- Refined and carefully analyzed by Minsky and Papert (1969) - their model is referred to as the **perceptron** model here

Perceptron



$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i \geq \theta$$

$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$

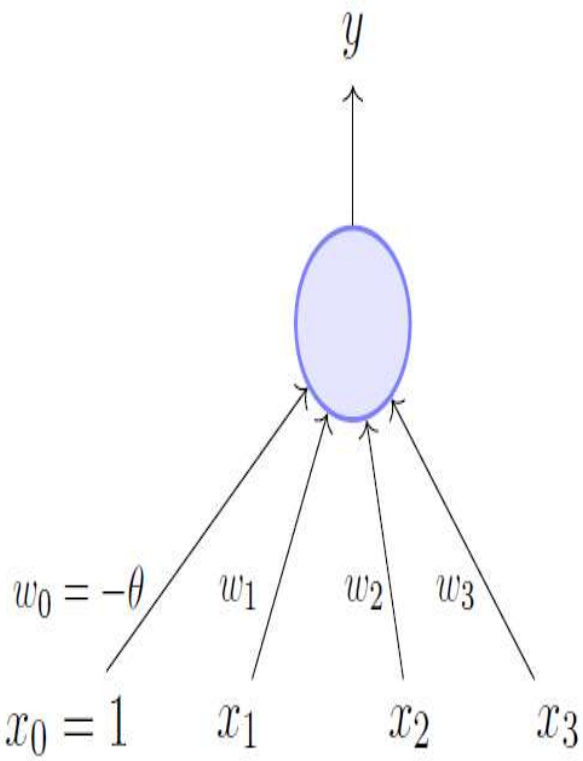
$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i - \theta < 0$$

A more accepted convention,

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

$$where, \quad x_0 = 1 \quad and \quad w_0 = -\theta$$

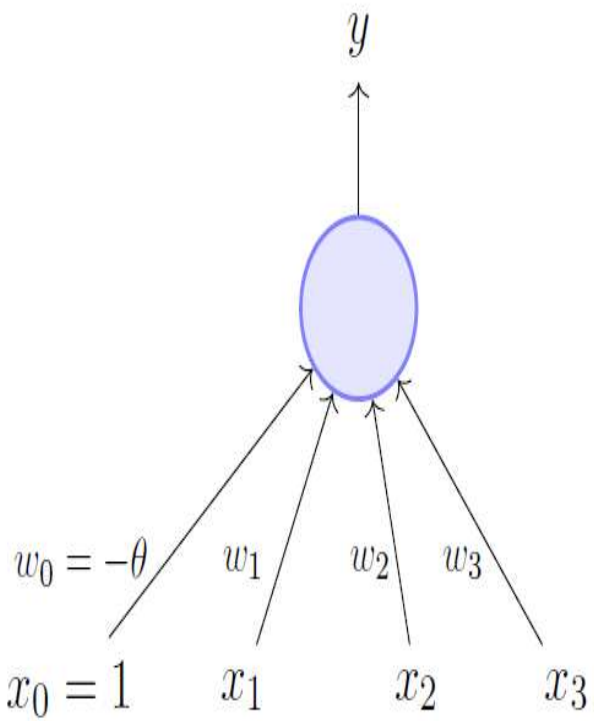


- Consider the task of predicting whether we would like a movie or not
- Suppose, we base our decision on 3 inputs (binary, for simplicity)
- Based on our past viewing experience (**data**), we may give a high weight to *isDirectorNolan* as compared to the other inputs
- Specifically, even if the actor is not *Matt Damon* and the genre is not *thriller* we would still want to cross the threshold θ by assigning a high weight to *isDirectorNolan*

$$x_1 = isActorDamon$$

$$x_2 = isGenreThriller$$

$$x_3 = isDirectorNolan$$



- w_0 is called the bias as it represents the prior (prejudice)
- A movie buff may have a very low threshold and may watch any movie irrespective of the genre, actor, director $[\theta = 0]$
- On the other hand, a selective viewer may only watch thrillers starring Matt Damon and directed by Nolan $[\theta = 3]$
- The weights $(w_1, w_2, ..., w_n)$ and the bias (w_0) will depend on the data (viewer history in this case)

$$x_1 = isActorDamon$$

$$x_2 = isGenreThriller$$

$$x_3 = isDirectorNolan$$

MP Neuron and Perceptron

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

$$y = 1 \quad if \sum_{i=0}^{n} x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} x_i < 0$$

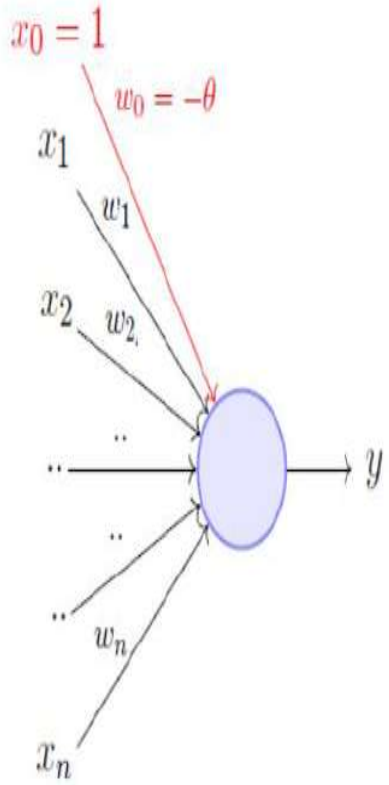
Perceptron

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

- From the equations it should be clear that even a perceptron separates the input space into two halves
- All inputs which produce a 1 lie on one side and all inputs which produce a 0 lie on the other side
- In other words, a single perceptron can only be used to implement linearly separable functions
- Then what is the difference? The weights (including threshold) can be learned and the inputs can be real valued
- We will first revisit some boolean functions and then see the perceptron learning algorithm (for learning weights)

Perceptron



A more accepted convention,

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

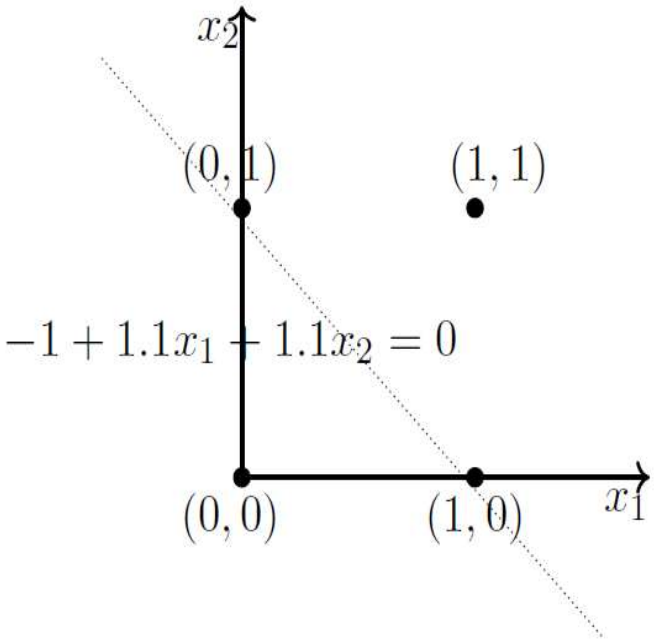
$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

$$where, \quad x_0 = 1 \quad and \quad w_0 = -\theta$$

A single perceptron can only be used to implement **linearly separable** functions. It takes both real and Boolean inputs and associates a set of **weights** to them, along with a **bias.**

OR function(Perceptron)

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$



$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 \geq -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 \geq -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 \geq -w_0$$

- One possible solution to this set of inequalities is $w_0 = -1, w_1 = 1.1, , w_2 = 1.1$ (and various other solutions are possible)

Algorithm: Perceptron Learning Algorithm

$P \leftarrow inputs \quad with \quad label \quad 1;$

$N \leftarrow inputs \quad with \quad label \quad 0;$

Initialize $\mathbf{w}$ randomly;

while $!convergence$ **do**

  Pick random $\mathbf{x} \in P \cup N$;

  if $\mathbf{x} \in P \quad and \quad \sum_{i=0}^{n} w_i * x_i < 0$ **then**

    $\mathbf{w} = \mathbf{w} + \mathbf{x}$;

  end

  if $\mathbf{x} \in N \quad and \quad \sum_{i=0}^{n} w_i * x_i \geq 0$ **then**

    $\mathbf{w} = \mathbf{w} - \mathbf{x}$;

  end

end

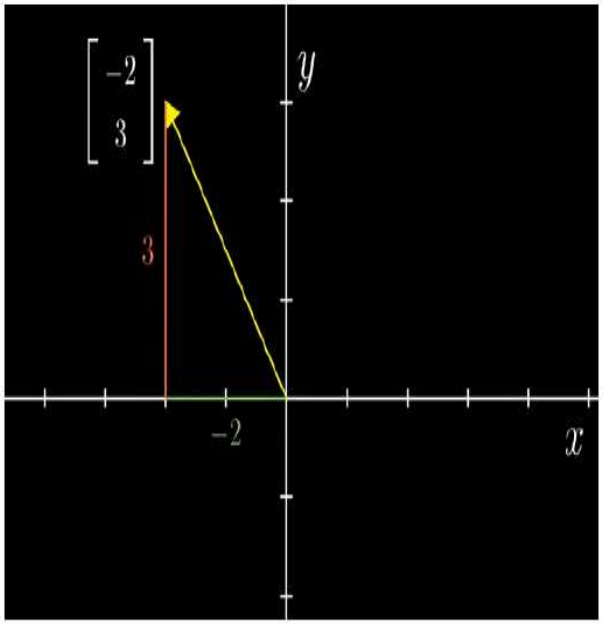
//the algorithm converges when all the inputs are classified correctly

Basics of Linear Algebra

A vector is anything that sits anywhere in space, has a magnitude and a direction. For CS guy, a vector is just a data structure used to store some data

Vector Representations

A 2-dimensional vector can be represented on a 2D plane as



Dot Product Of Two Vectors

The dot product of these vectors (w.x) could be computed as follows:

$$\mathrm{w} = [w_0, w_1, w_2, ..., w_n]$$

$$\mathrm{x} = [1, x_1, x_2, ..., x_n]$$

$$\mathrm{w} \cdot \mathrm{x} = \mathrm{w}^{\mathrm{T}}\mathrm{x} = \sum_{i=0}^{n} w_i * x_i$$

The decision boundary line which a perceptron gives out that separates positive examples from the negative ones is really just w . x = 0.

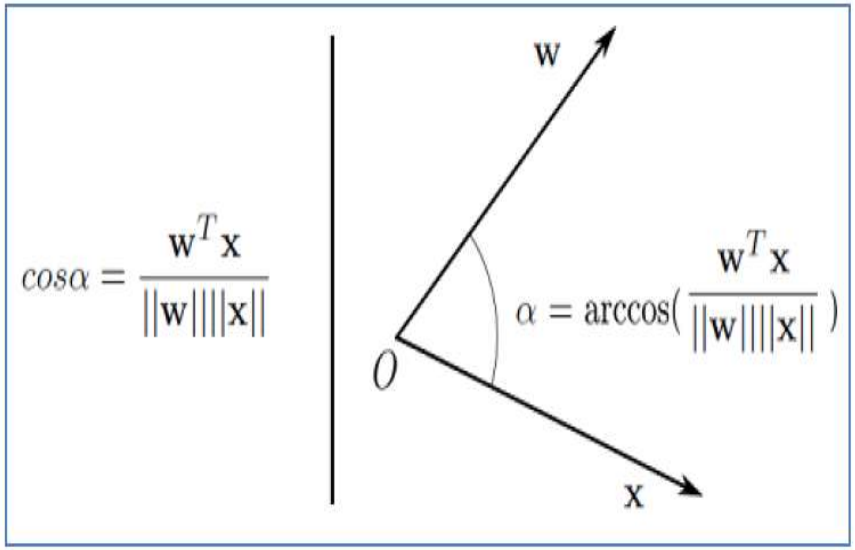
Basics of Linear Algebra

Angle Between Two Vectors

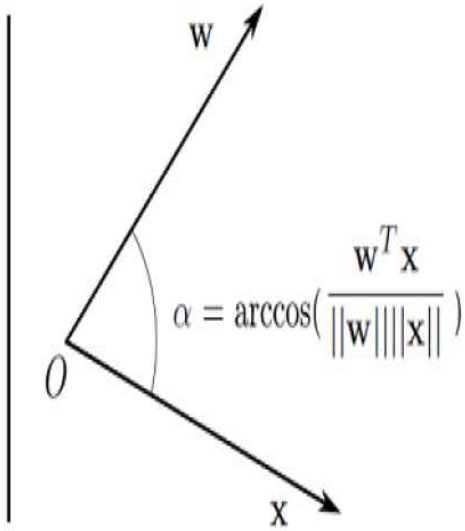
The angle between the vectors and their individual magnitudes

$$\mathbf{w}^T\mathbf{x} = \|\mathbf{w}\|\|\mathbf{x}\| cos\alpha$$

$$cos\alpha = \frac{\mathbf{w}^T\mathbf{x}}{\|\mathbf{w}\|\|\mathbf{x}\|}$$



$$cos\alpha = \frac{\mathbf{w}^T\mathbf{x}}{\|\mathbf{w}\|\|\mathbf{x}\|}$$



Why would this work?

When x belongs to P, we want w.x > 0, basic perceptron rule. i.e when x belongs P, the angle between w and x should be less than 90 degrees? The angle between and x should be less than 90 because the cosine of the angle is proportional to th dot product. So whatever the w vector may be, as long as it makes an angle le than 90 degrees with the positive example data vectors (x E P) and an angle mo than 90 degrees with the negative example data vectors (x E N).

So whatever the w vector may be, as long

as it makes an angle less than 90 degrees

with the positive example data vectors (x E P)

and an angle more than 90 degrees

with the negative example data vectors (x E N)

$$cos\alpha = \frac{\mathbf{w}^T\mathbf{x}}{||\mathbf{w}||||\mathbf{x}||} \quad \Big| \quad cos\alpha \propto \mathbf{w}^T\mathbf{x}$$

So if $\mathbf{w}^T\mathbf{x} > 0 \Rightarrow cos\alpha > 0 \Rightarrow \alpha < 9$

Similarly, if $\mathbf{w}^T\mathbf{x} < 0 \Rightarrow cos\alpha < 0 \Rightarrow \alpha > 9$

Perceptron learning algorithm

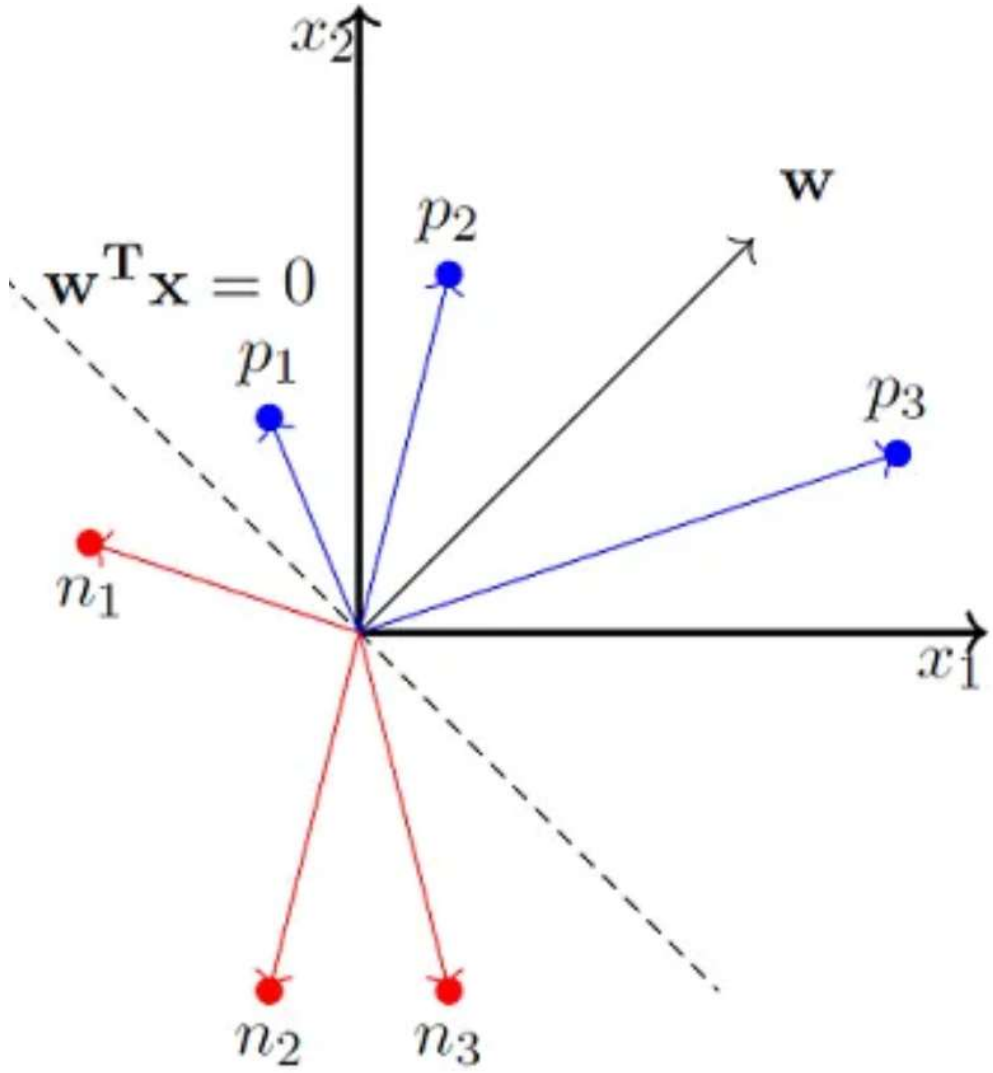
(α_{new}) when $\mathbf{w_{new}} = \mathbf{w} + \mathbf{x}$

$$cos(\alpha_{new}) \propto \mathbf{w_{new}}^T \mathbf{x}$$
$$\propto (\mathbf{w} + \mathbf{x})^T \mathbf{x}$$
$$\propto \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}$$
$$\propto cos\alpha + \mathbf{x}^T \mathbf{x}$$
$$cos(\alpha_{new}) > cos\alpha$$

(α_{new}) when $\mathbf{w_{new}} = \mathbf{w} - \mathbf{x}$

$$cos(\alpha_{new}) \propto \mathbf{w_{new}}^T \mathbf{x}$$
$$\propto (\mathbf{w} - \mathbf{x})^T \mathbf{x}$$
$$\propto \mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x}$$
$$\propto cos\alpha - \mathbf{x}^T \mathbf{x}$$
$$cos(\alpha_{new}) < cos\alpha$$

Perceptron learning algorithm



Problems

Number of sides=3	Equal sides	Equilateral triangle
False	false	false
False	true	false
True	false	false
True	true	true

Implement the above table with appropriate logic function using any neural network model.

If a geometric shape has 3 sides and are equal, then it is a equilateral triangle.

The sentence is true only if both the number of sides=3 and Equal sides should be true, otherwise it is false. So this can be implemented with a AND function as it returns a true value only if both the inputs are true, else it returns a false value. We can use MP neuron model for implementing this.'1' represents true value and '0' represents false value.

Number of sides=3(X1)	Equal sides (X2)	Equilateral triangle (Y)
0	0	0
0	1	0
1	0	0
1	1	1

Draw its perceptron model

Problems

Consider the unit shown on Figure 1. Suppose that the weights corresponding to the three inputs have the following values:

$$\boxed{\begin{array}{lcr} w_1 & = & 2 \\ w_2 & = & -4 \\ w_3 & = & 1 \end{array}}$$

and the activation of the unit is given by the step–function:

$$\varphi(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Calculate what will be the output value y of the unit for each of the following input patterns:

Pattern	P_1	P_2	P_3	P_4
x_1	1	0	1	1
x_2	0	1	0	1
x_3	0	1	1	1

Answer: *To find the output value y for each pattern we have to:*

a) *Calculate the weighted sum:* $v = \sum_i w_i x_i = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3$

b) *Apply the activation function to v*

Problems

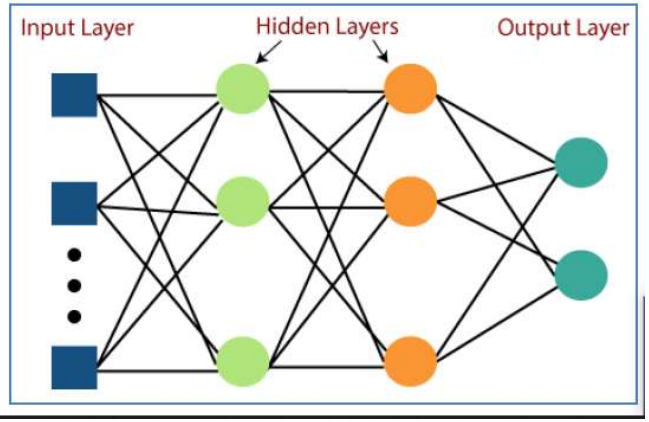
Consider a single perceptron with sign activation function.

The perceptron is represented by weight vector $[0.4 - 0.3\ 0.1]^t$ and a bias $\theta = 0$. If the input vector to the perceptron is X = [0.2 0.6 0.5] then the output of the perceptron is :

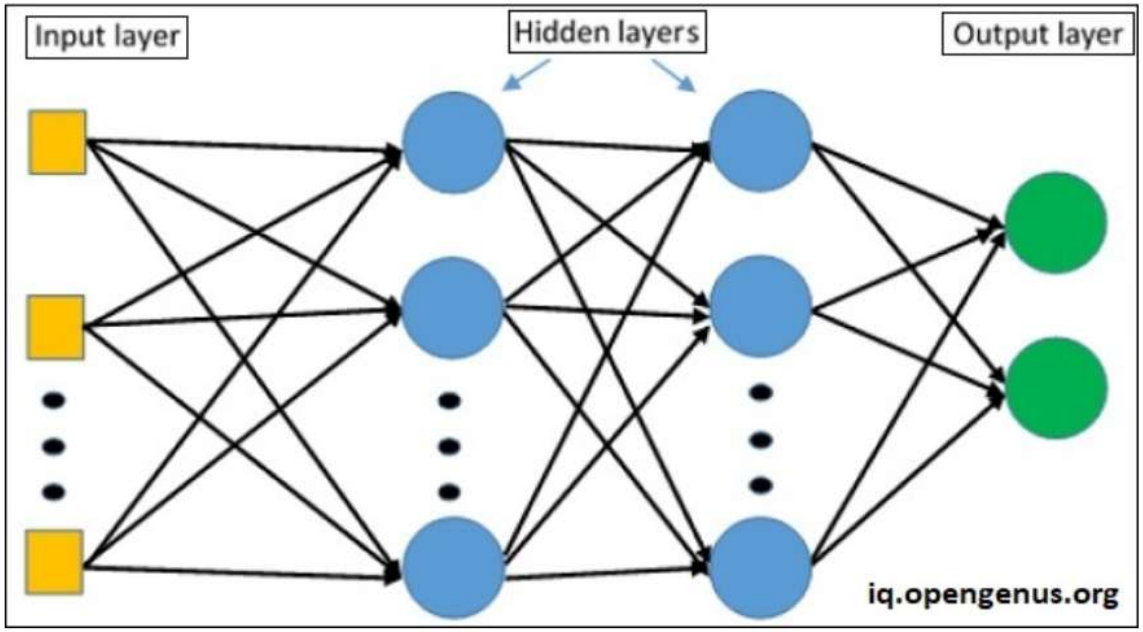
Multilayer Perceptron

Multi-Layer perceptron defines the most complex architecture of artificial neural networks. It is substantially formed from multiple layers of the perceptron. MLP networks are used for supervised learning format. A typical learning algorithm for MLP networks is also called **back propagation's algorithm.**

A multilayer perceptron (MLP) is a feed forward artificial neural network that generates a set of outputs from a set of inputs. An MLP is characterized by several layers of input nodes connected as a directed graph between the input nodes connected as a directed graph between the input and output layers. MLP uses backpropagation for training the network.



Multi Layered Perceptron



•It has many layers of nodes (known as artificial neurons) that connect to each other.

The architecture of an MLP consists of three key components: the input layer, hidden layer(s), and output layer.

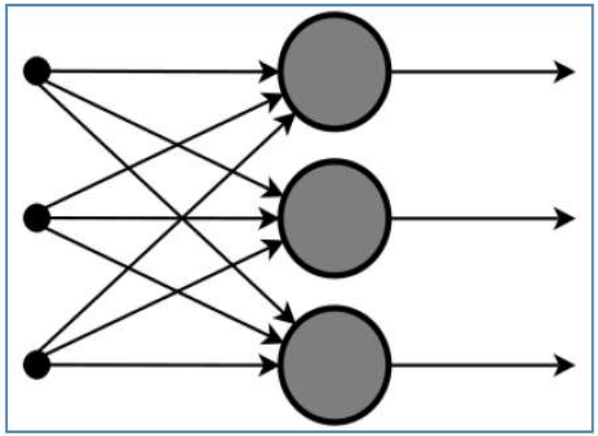
Input Layer: The input layer receives the input data and passes it to the hidden layer(s). Input layer consists of nodes, each of which corresponds to a feature in the input data.

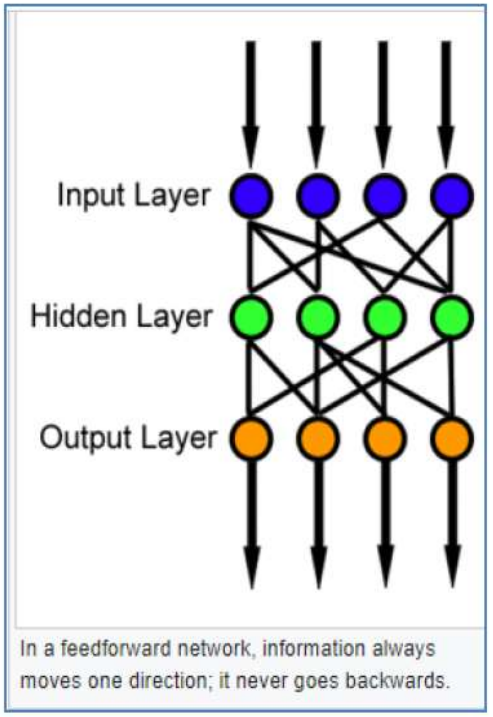
MLP

- Hidden Layer(s): The hidden layer(s) are responsible for transforming the input data into a suitable representation for the output layer.
- They consist of nodes, which are connected to the input layer and other hidden layer(s) through weights and biases.
- The number of hidden layer(s) and the number of nodes in each layer can be adjusted to optimize performance for a specific task.
- Common activation functions used in hidden layers include sigmoid, tanh, and rectified linear unit (ReLU).
- Output Layer: The output layer receives the transformed representation of the input data from the hidden layer(s) and generates the final output.
- It consists of nodes, each of which corresponds to a class or a continuous value, depending on the task.
- The output layer uses a specific activation function that is appropriate for the task, such as softmax for classification and linear for regression.

Feed forward Neural Networks.

- Below is a Single layer feed-forward network. Here, the sum of the products of inputs and weights are calculated and fed to the output. The output is considered if it is above a certain value i.e threshold and the neuron fires with an activated output and if it does not fire, the deactivated value is emitted.



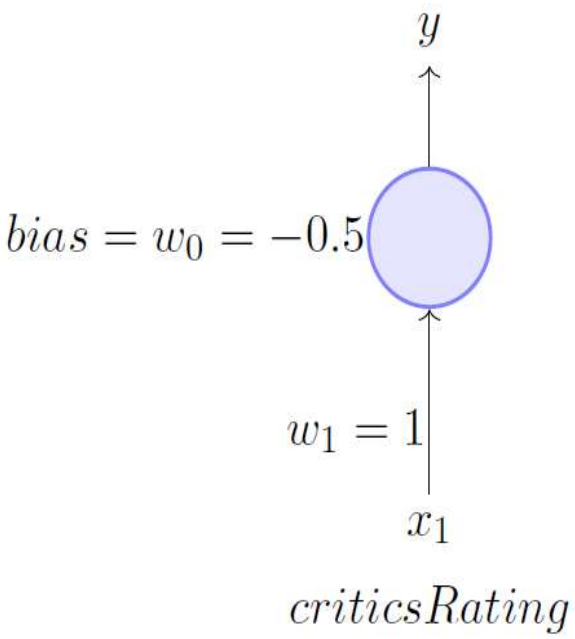


In a feedforward network, information always moves one direction; it never goes backwards.

Application of Feedforward neural networks are found in computer vision and speech recognition where classifying the target classes is complicated.

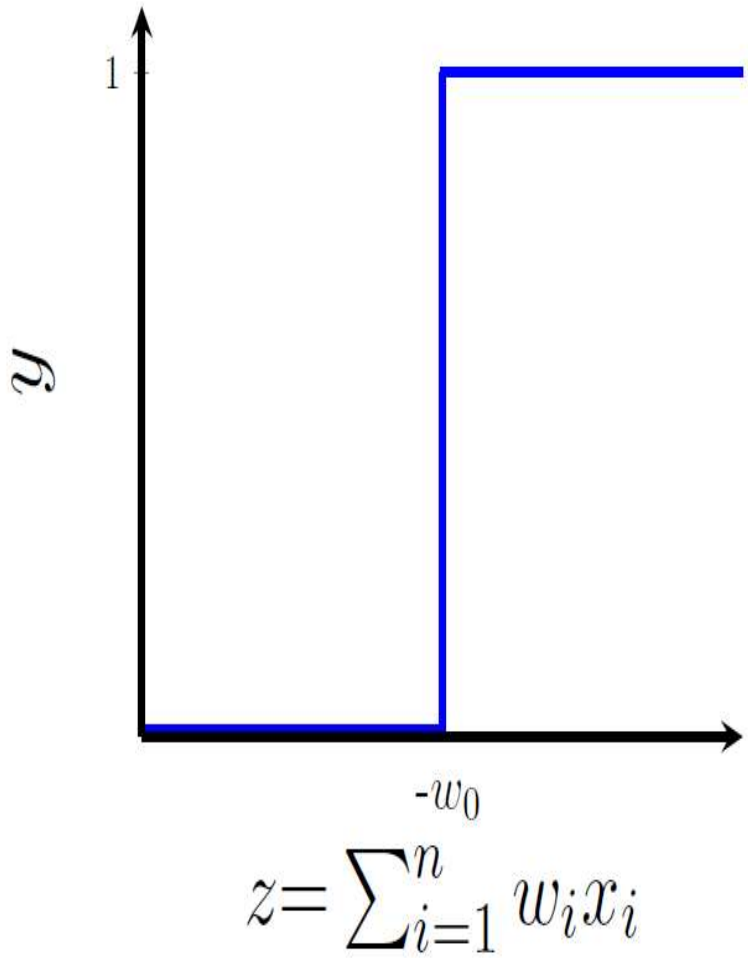
Networks in which the input signal is fed continuously in one direction from connection to connection until it reaches the output layer are called **feedforward networks.**

Perceptron



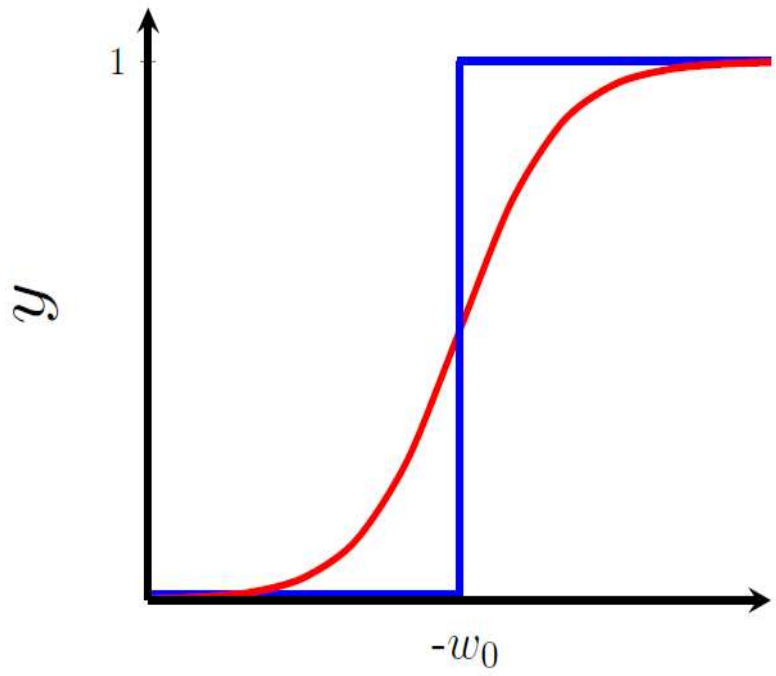
- The thresholding logic used by a perceptron is very harsh !
- For example, let us return to our problem of deciding whether we will like or dislike a movie
- Consider that we base our decision only on one input ($x_1 = criticsRating$ which lies between 0 and 1)
- If the threshold is 0.5 ($w_0 = -0.5$) and $w_1 = 1$ then what would be the decision for a movie with $criticsRating = 0.51$? (like)
- What about a movie with $criticsRating = 0.49$? (dislike)
- It seems harsh that we would like a movie with rating 0.51 but not one with a rating of 0.49

Perceptron



- This behavior is not a characteristic of the specific problem we chose or the specific weight and threshold that we chose
- It is a characteristic of the perceptron function itself which behaves like a step function
- There will always be this sudden change in the decision (from 0 to 1) when $\sum_{i=1}^{n} w_i x_i$ crosses the threshold ($-w_0$)
- For most real world applications we would expect a smoother decision function which gradually changes from 0 to 1

Sigmoid Neuron



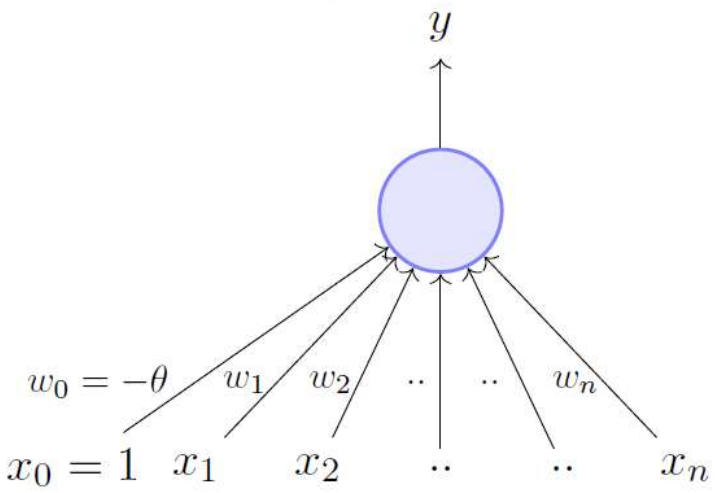
- Introducing sigmoid neurons where the output function is much smoother than the step function
- Here is one form of the sigmoid function called the logistic function

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^{n} w_i x_i)}}$$

- We no longer see a sharp transition around the threshold -w_0
- Also the output y is no longer binary but a real value between 0 and 1 which can be interpreted as a probability
- Instead of a like/dislike decision we get the probability of liking the movie

Sigmoid Neuron

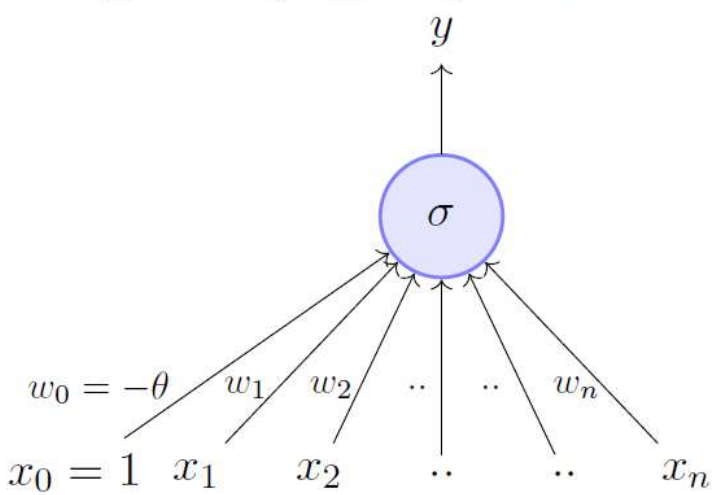
Perceptron



$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

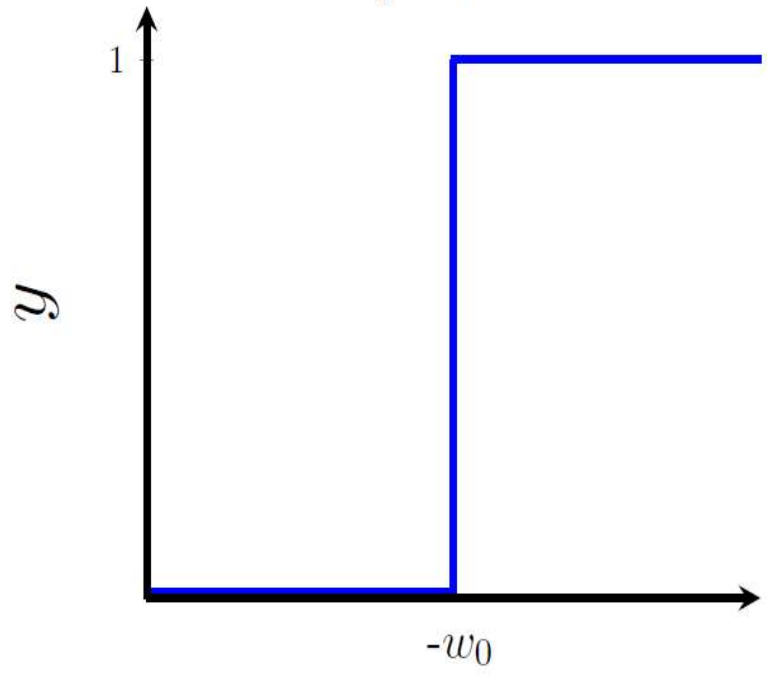
Sigmoid (logistic) Neuron



$$y = \frac{1}{1 + e^{-(\sum_{i=0}^{n} w_i x_i)}}$$

Sigmoid Neuron

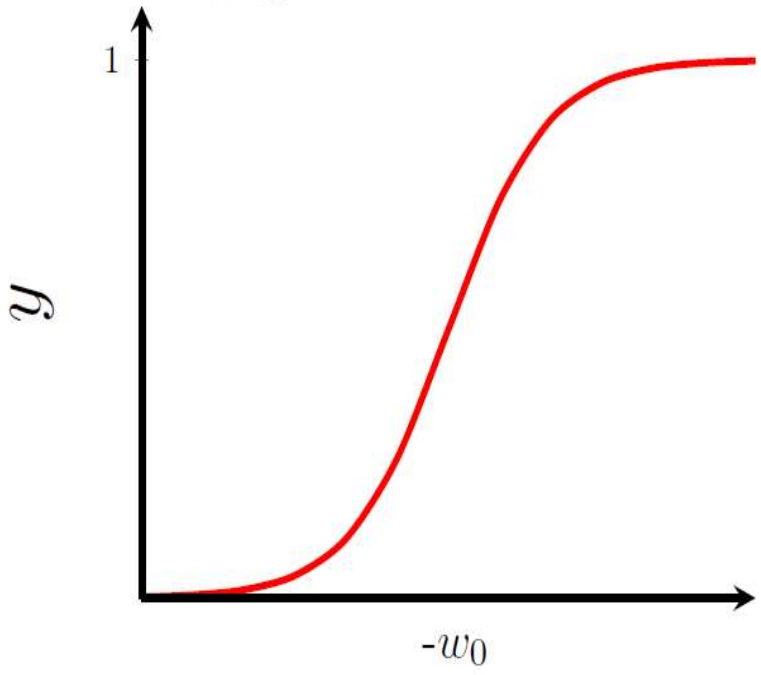
Perceptron



$$z = \sum_{i=1}^{n} w_i x_i$$

Not smooth, not continuous (at $w0$), **not differentiable**

Sigmoid Neuron



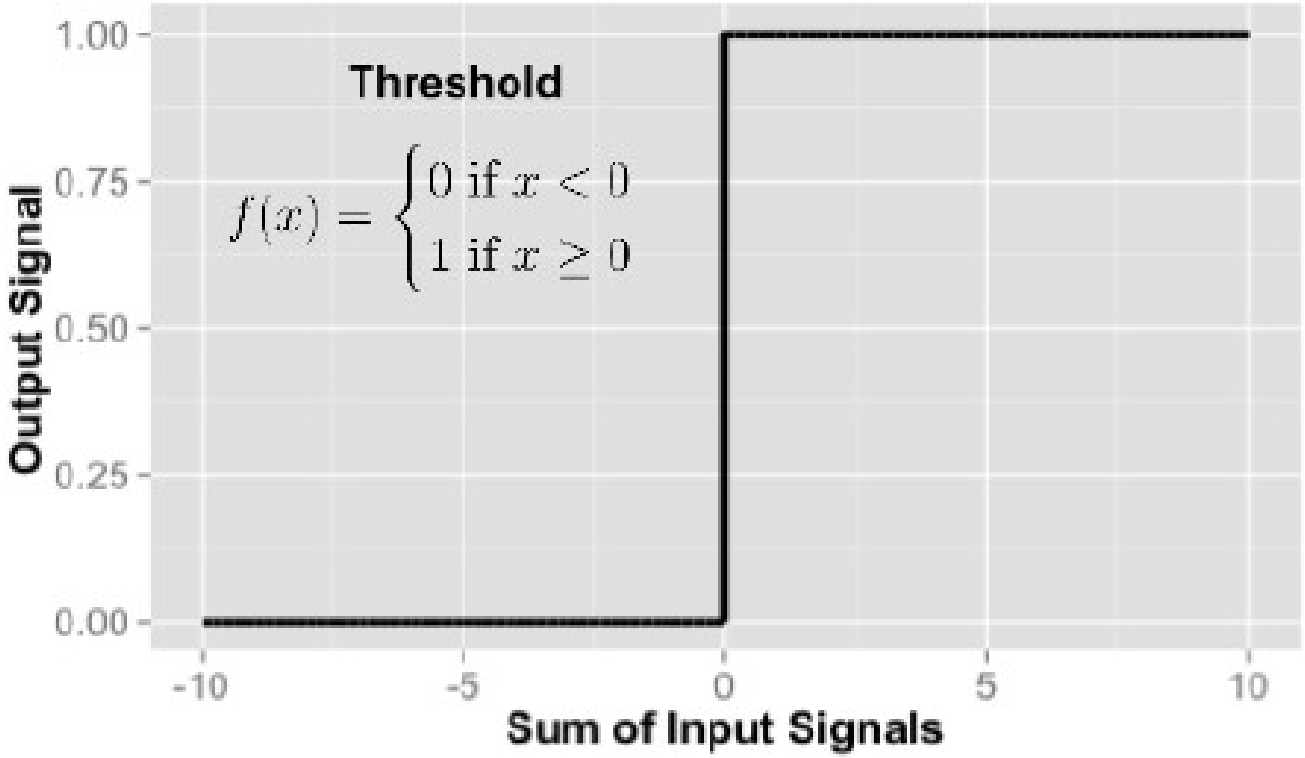
$$z = \sum_{i=1}^{n} w_i x_i$$

Smooth, continuous, **differentiable**

Activation functions

- The activation function is **the mechanism** by which the artificial neuron **processes information and passes** it throughout the network.
- It is a process that involves summing the total input signal and determining whether it meets the firing threshold.
- If so, the neuron passes on the signal; otherwise, it does nothing.
- In ANN terms, this is known as a **threshold activation function,** as it results in an output signal only once a specified input threshold has been attained.

Unit Step function



- In this case, the neuron fires when the sum of input signals is at least zero.
- Because of its shape, it is sometimes called a **unit step activation function.**

Other Activation functions

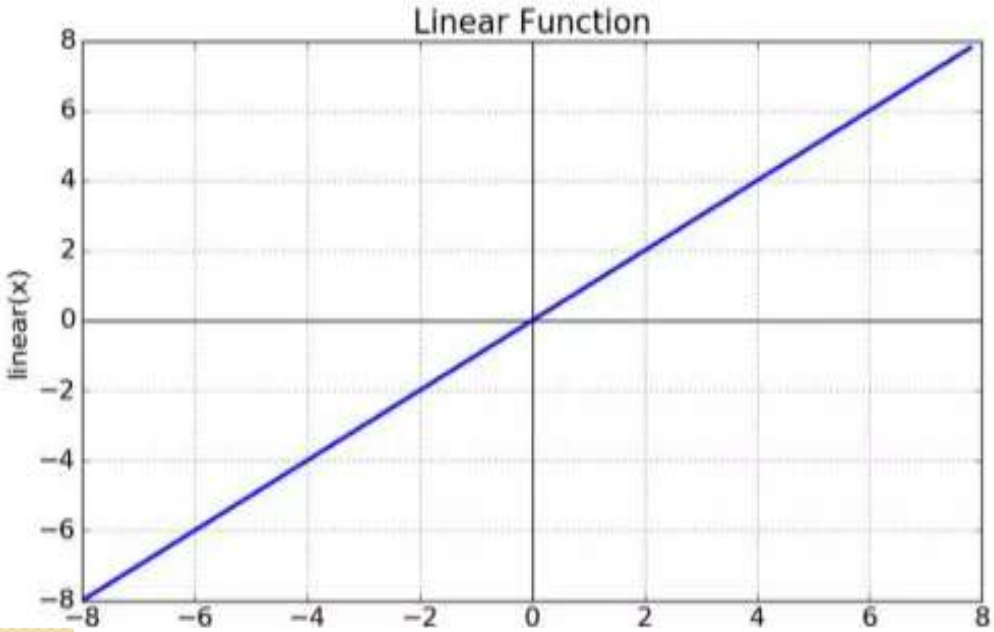
Types

- The Activation Functions can be basically divided into 2 types-
- Linear Activation Function

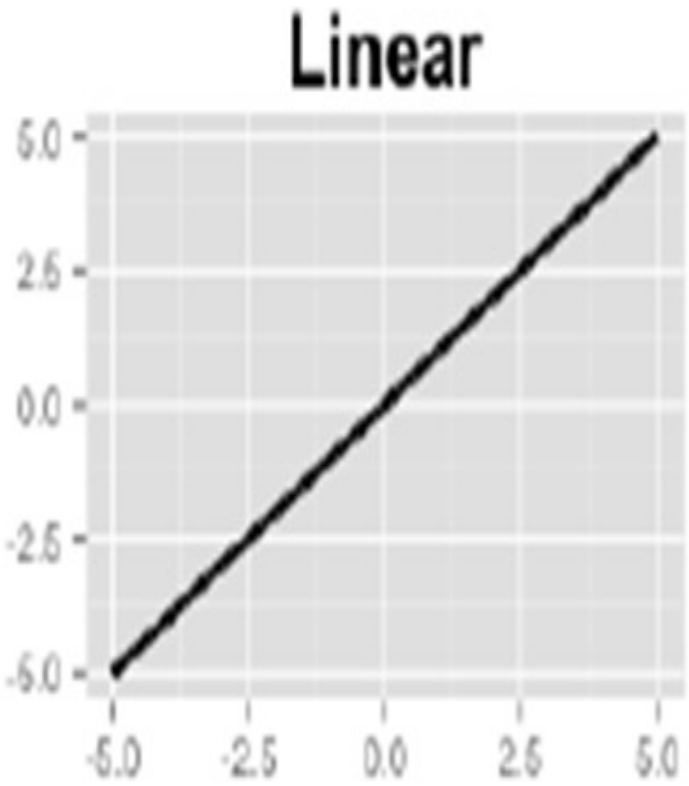
- Non-linear Activation Functions

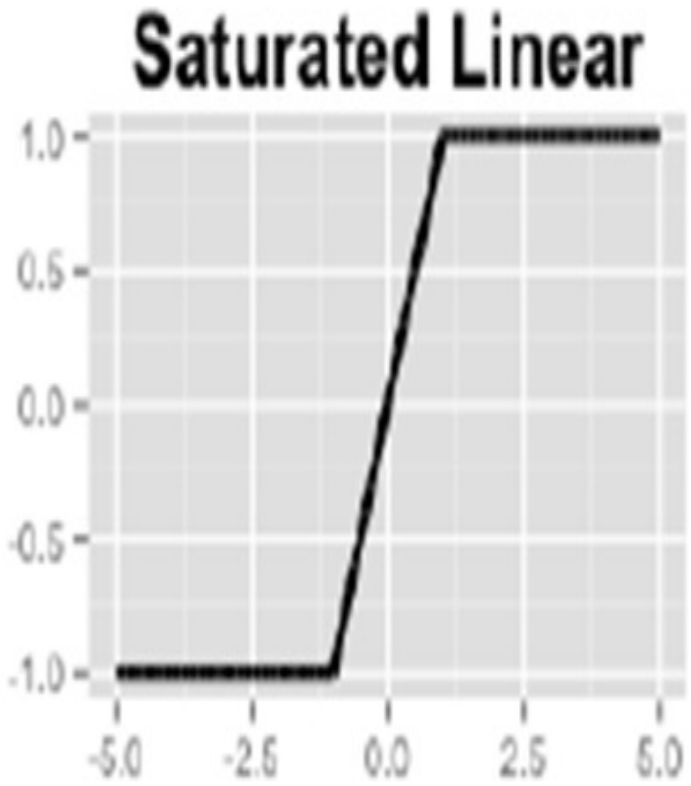
Linear or Identity Activation Function

- As you can see the function is a line or linear. Therefore, the output of the functions will not be confined between any range.
- **Equation :** f(x) = x
- **Range :** (-infinity to infinity)
- It doesn't help with the complexity or various parameters of usual data that is fed to the neural networks.



Linear Activation function

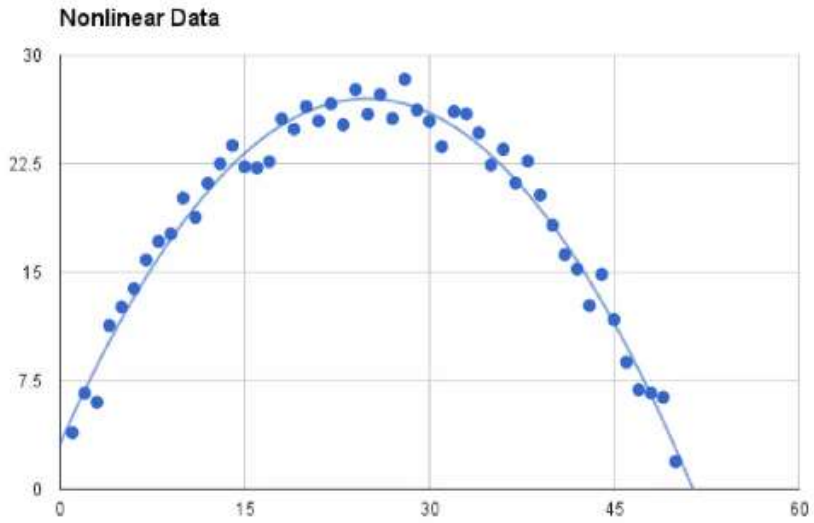




Nonlinear Activation function

Non-linear Activation Function

- It makes it easy for the model to generalize or adapt with variety of data and to differentiate between the output.



Nonlinear Activation function

- Important nonlinear activation functions are
  - Sigmoid
  - Tanh
  - Relu
  - Leaky ReLu
  - Softmax

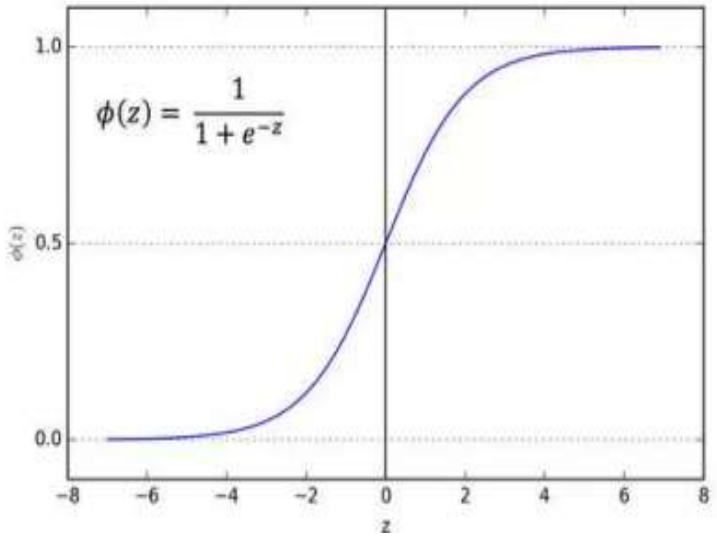
Sigmoid Activation function

The Nonlinear Activation Functions are mainly divided on the basis of their **range or curves**-

- **Sigmoid or Logistic Activation Function**
- The Sigmoid Function curve looks like a S-shape.
- The main reason why we use sigmoid function is because it exists between **(0 to 1).** Therefore, it is especially used for models where we have to **predict the probability** as an output.Since probability of anything exists only between the range of **0 and 1,** sigmoid is the right choice.
- The logistic sigmoid function can cause a neural network to get stuck at the training time.

The beauty of an exponent is that the value never reaches zero nor exceed 1 in the above equation.

The large negative numbers are scaled towards 0 and large positive numbers are scaled towards 1.



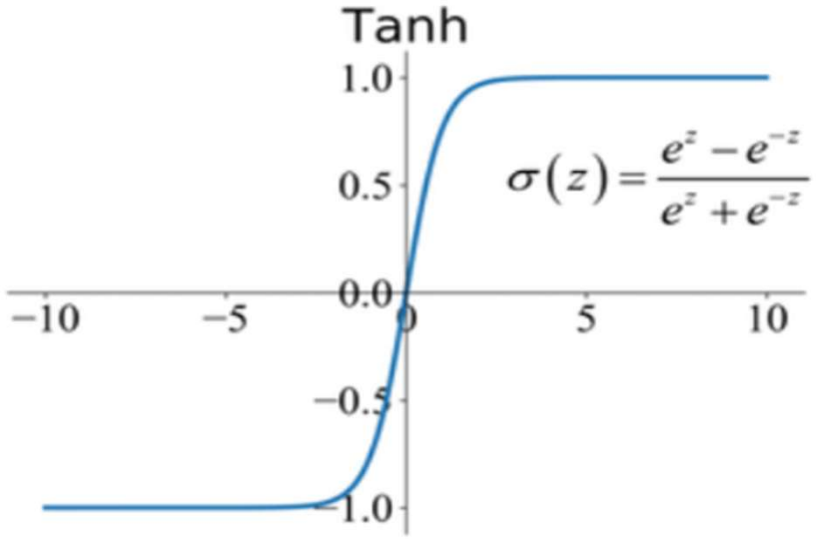
Sigmoid activation function

- It is the most commonly used **activation function.**
- e *is the base* of natural logarithms.
- S shape
- output values can fall anywhere in the range from 0 to 1
- the sigmoid is **differentiable**
- which means that it is possible to calculate the derivative across the entire range of inputs.

Tanh (Hyperbolic Tangent)

- **Tanh Activation** is an activation function used for neural networks(especially in hidden layers)
- Tanh function became preferred over the sigmoid function as it gave better performance for multi-layer neural networks

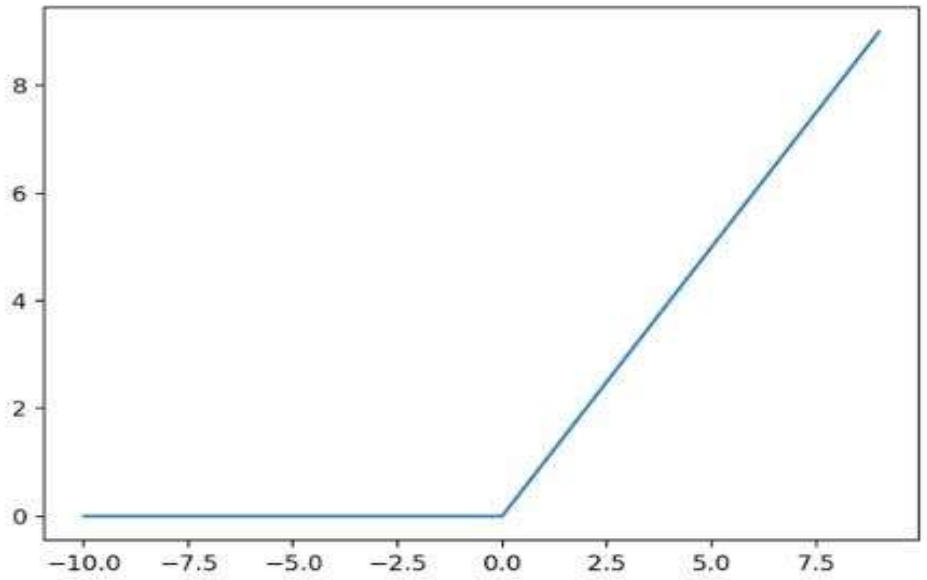
$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



ReLU

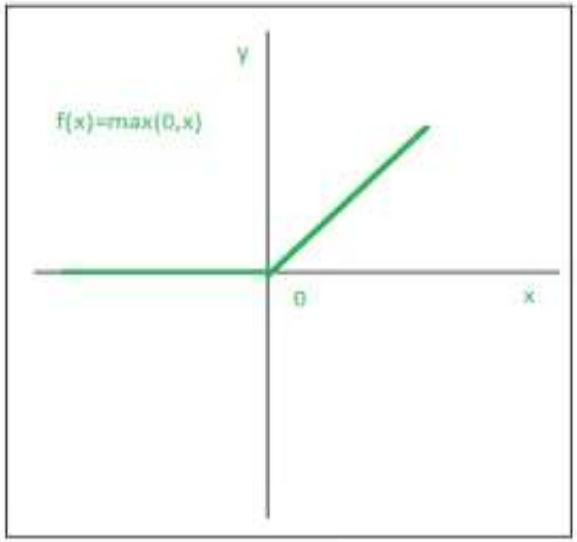
- **Rectified Linear Units**, or **ReLUs**, are a type of activation function that are linear in the positive dimension, but zero in the negative dimension.

f(x)=max(0,x)



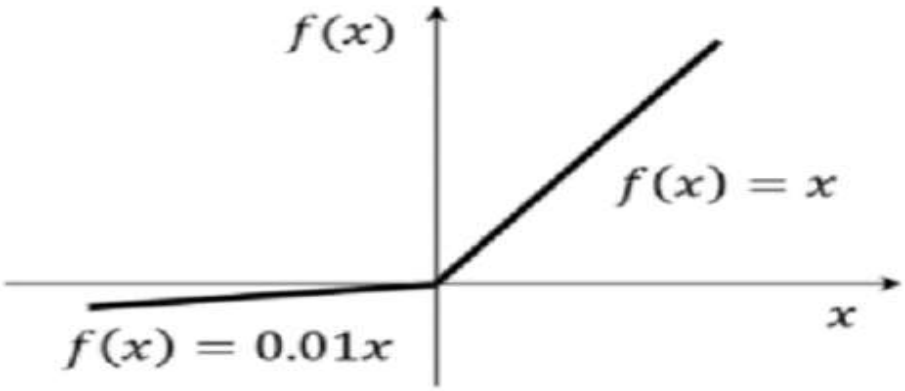
ReLu- Rectified Linear units

- It was recently proved that it had *6 times improvement in convergence* from Tanh function.
- *R(x) = max(0,x) i.e if x < 0 , R(x) = 0 and if x >= 0 , R(x) = x.*
- Hence it avoids and rectifies **vanishing gradient** problem.
- But its limitation is that it should only be used within Hidden layers of a Neural Network Model.
- Another problem with ReLu is that some gradients can be fragile during training and can die. It can cause a weight update which will makes it never activate on any data point again. Simply saying that ReLu could result in Dead Neurons.



Leaky RELU

Equation: $f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$ where α is a small constant (e.g., 0.01)



LeakyReLU activation function

Characteristics:

- **Output Range:** (-∞, ∞)
- **Pros:** Mitigates "dying ReLU" problem
- **Cons:** Choice of α is arbitrary and requires tuning

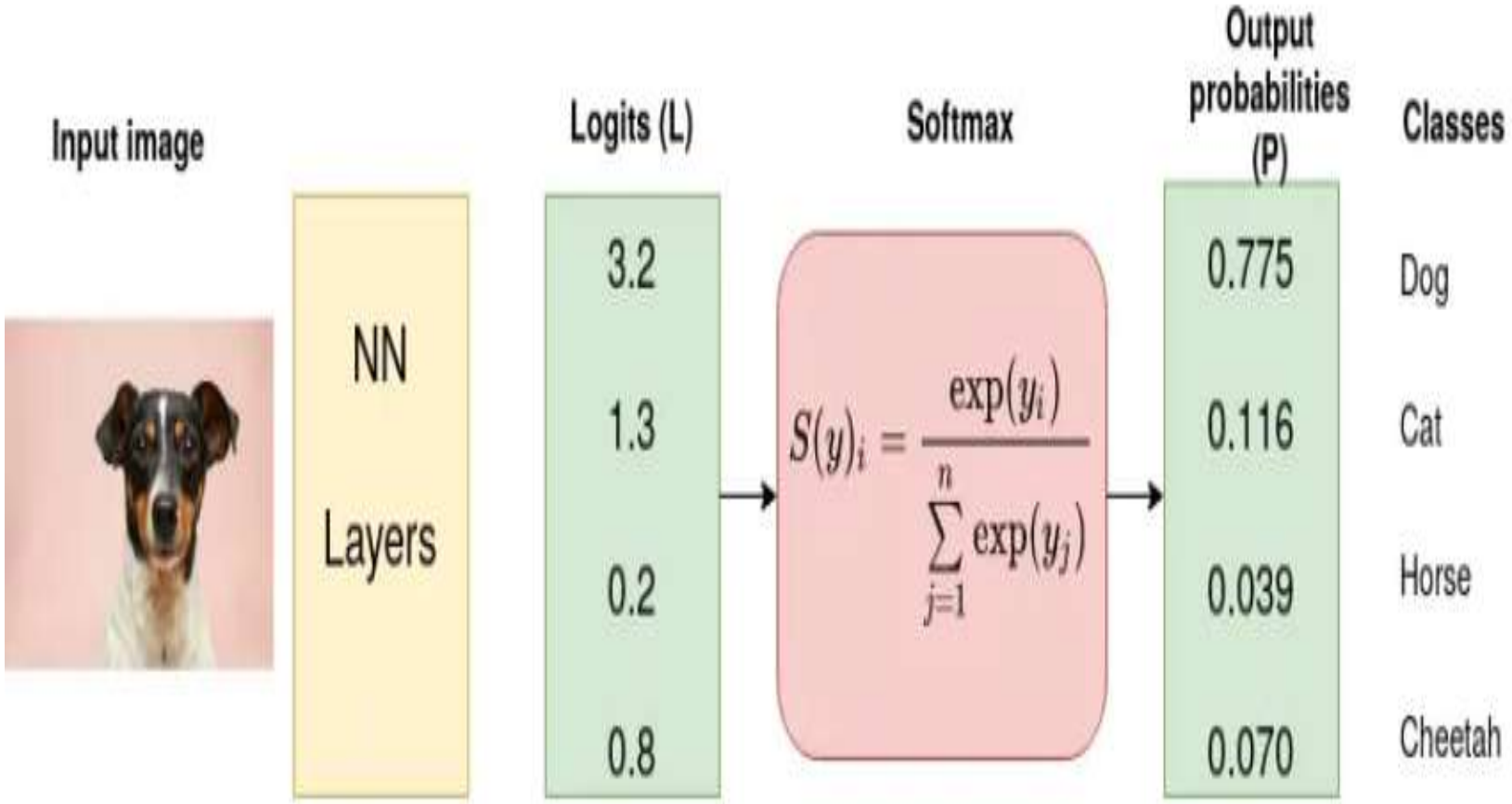
Softmax function

- It is used for multiclass classification.
- It is usually placed as the last layer in the deep learning model.
- The output of a Softmax is a vector (say y) with probabilities of each possible outcome. The probabilities in vector y sums to one for all possible outcomes or classes.
- Mathematically, Softmax is defined as,

$$S(y)_i = \frac{\exp(y_i)}{\sum_{j=1}^{n} \exp(y_j)}$$

y	is an input vector to a softmax function, S. It consist of n elements for n classes (possible outcomes)
y_i	the i-th element of the input vector. It can take any value between -inf and +inf

Softmax Function



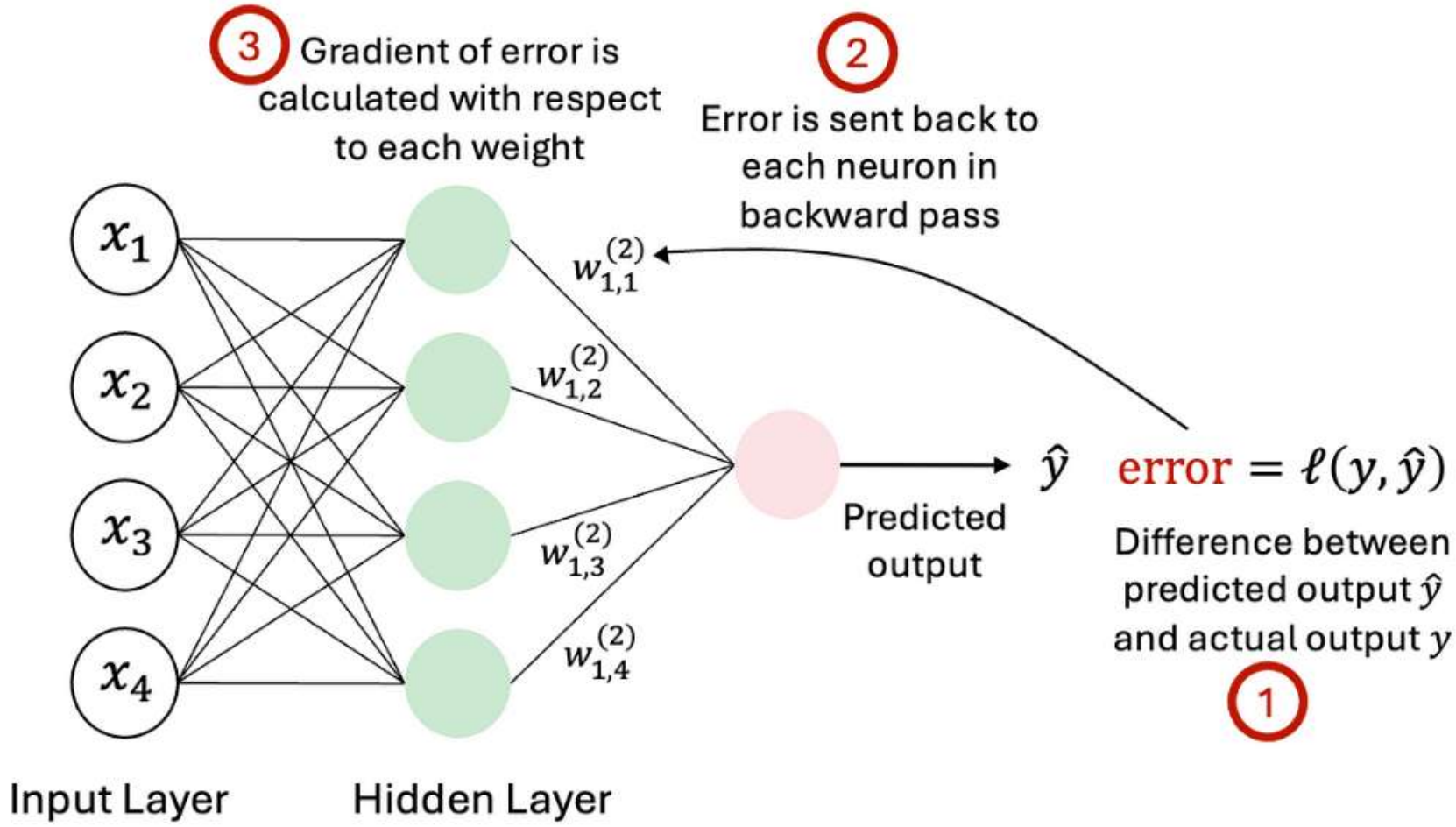
Characteristics of neural networks

1. An **activation function,** which transforms a neuron's net input signal into a single output signal.
2. A **network topology (or architecture),** which describes the number of neurons in the model as well as the number of layers and the manner in which they are connected.
3. The **training algorithm** that specifies how connection weights are set in order to excite neurons.

Training algorithm

- Like a newborn child, ANN must be trained with experience.
- As it processes the input data, connections between the neurons are strengthened or weakened.
- Mainly 2 algorithms for learning a single perceptron.
  - Perceptron rule: used when training dataset is linearly separable.
  - Delta rule: used when training dataset is not linearly separable.

Back Propagation



Backpropagation

- The training algorithm, which used a strategy of back-propagating errors, is now known as **backpropagation.**
- slow relative to many other machine learning algorithms
- **Multi layer feed forward networks that uses the backpropagation** algorithm are common now.

Outline of the algorithm

1. Initially the weights are assigned at random.

2. Then the algorithm iterates through many cycles of two processes until a stopping criterion is reached. Each cycle is known as an epoch. Each epoch includes:
   (a) A forward phase in which the neurons are activated in sequence from the input layer to the output layer, applying each neuron's weights and activation function along the way. Upon reaching the final layer, an output signal is produced.
   (b) A backward phase in which the network's output signal resulting from the forward phase is compared to the true target value in the training data. The difference between the network's output signal and the true value results in an error that is propagated backwards in the network to modify the connection weights between neurons and reduce future errors.

Outline of the algorithm (Cont'd)

3. The technique used to determine how much a weight should be changed is known as gradient descent method. At every stage of the computation, the error is a function of the weights. If we plot the error against the weights , we get a higher dimensional analog of something like a curve or surface.

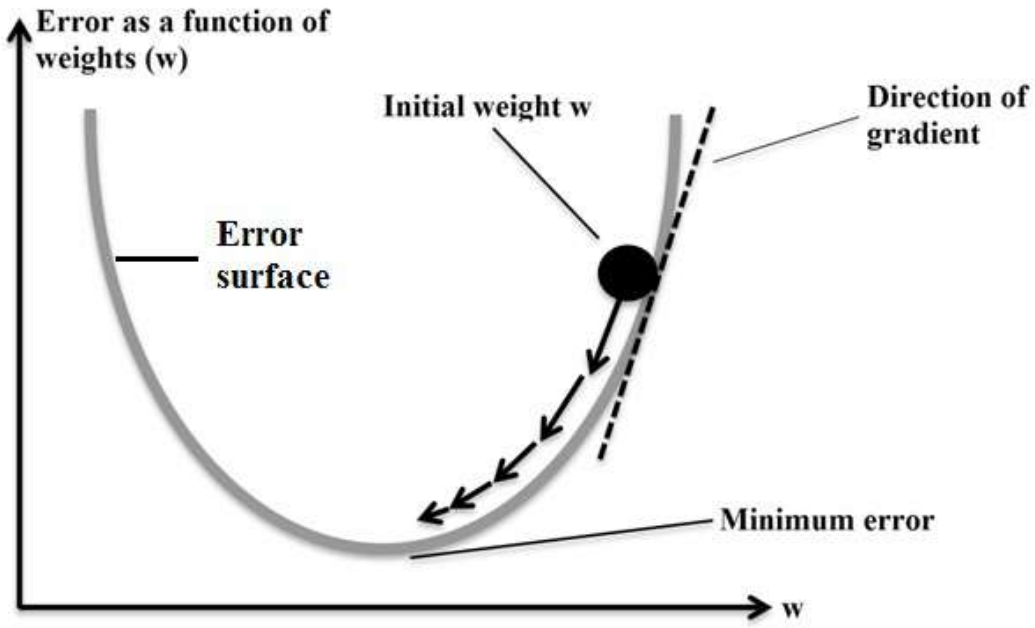
- At any point on this surface, the gradient suggests how steeply the error will be reduced or increased for a change in the weight. The algorithm will attempt to change the weights that result in the greatest reduction in error

Note: The **back propagation algorithm** uses the **derivative of each neuron's activation function** to identify the **gradient** in the direction of each of the incoming weights.

Cost Function/Error Function/Loss Function

- It is a function that measures the performance of a model for given data. Also called the loss function/ objective function/scoring function/error function.
- Let y be the output variable. $y_1, y_2, \ldots y_n$ are actual values of y. Also $\hat{y}_1, \hat{y}_2, \ldots\ldots \hat{y}_n$ are predicted values
- SSE : Sum of squares of differences between predicted and actual values of y denoted by $SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$ (test set; predicted vaue; actual value)
- MSE : Mean of the sum of squares of the differences between the predicted and actual values of y *denoted* $MSE = \frac{1}{n} \Sigma \left(y - \widehat{y} \right)^2$ (The square of the difference between actual and predicted)

A simplied model of the error surface showing the direction of gradient



Gradient Descent

- To minimize a cost function –Use **Gradient descent**.
- **Gradient descent** is an optimization algorithm **used** to minimize some function by iteratively moving in the direction of **steepest descent** as defined by the negative of the **gradient**. Here we **use gradient descent** to minimize the cost function(differences between actual y and predicted y).
- It attempts to find a local or global minima of a function.
- Gradient descent enables a model to learn the gradient or *direction* that the model should take in order to reduce errors

Gradient Descent

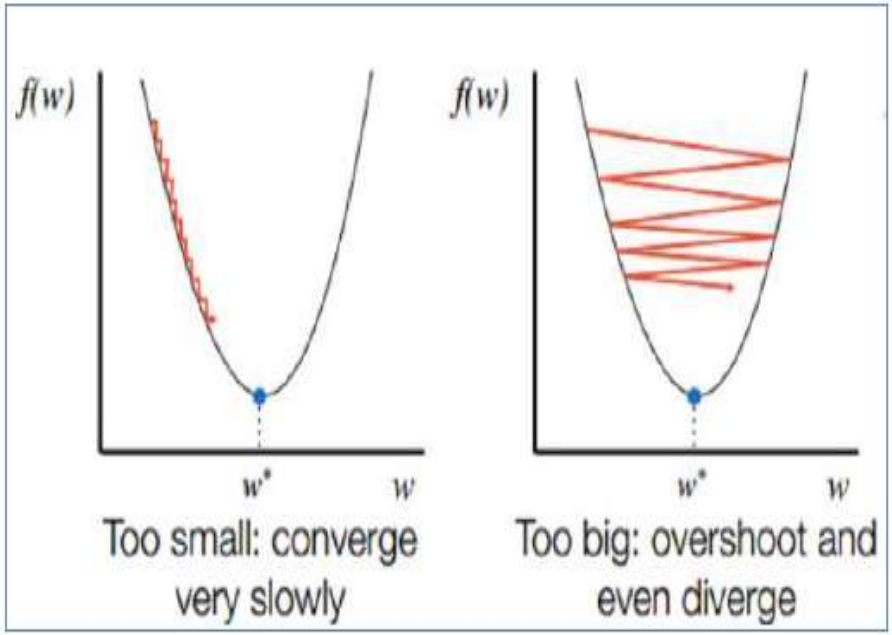
- The goal of the gradient Descent algorithm is to minimize the given function.

  1) Compute the gradient(slope), the first order derivative of the function at that point.

  2) Make a step in the direction opposite to the direction of the gradient.

- We have now in the direction to move in, next is to decide the size of the step.
- Learning rate (η)– A tuning parameter in the optimization process and it decides the length of the steps.

Gradient Descent



- If the learning rate is too high it might overshoot the minima and keep bouncing without reaching the minima.
- If the learning rate is too small the training might turn out to be too long.
- Choose an optimal learning rate so that model converges to the minimum.

Gradient Descent Algorithm

Gradient Descent method's steps are:

1. Choose a starting point (initialization)
2. Calculate gradient at this point
3. Make a scaled step in the opposite direction to the gradient

   (objective: minimize)
4. Repeat points 2 and 3 until one of the criteria is met:
5. Maximum number of iterations reached
6. Step size is smaller than the tolerance.

This function takes 5 parameters:

1. **starting point** - In practice, it is often a random initialisation
2. **gradient function** - has to be specified before-hand
3. **learning rate** - scaling factor for step sizes
4. maximum number of iterations
5. tolerance to conditionally stop the algorithm (a default value is 0.01)

Gradient Descent Algorithm

The objective learning algorithm is to determine the best possible values for the parameters(w & b), such that the overall loss of the model is minimized as such as possible.

Initialize w, b;

Iterate over data;

Compute $\hat{y}$

Compute L(w,b)

$W_{t+1} = W_t - \eta \Delta W_t$

$b_{t+1} = b_t - \eta \Delta b_t$

where $\Delta W_t = \frac{\partial L(W,b)}{\partial w}$ at $W=W_t, b=b_t$

$\Delta b_t = \frac{\partial L(W,b)}{\partial b}$ at $W=W_t, b=b_t$

Problem

Network Architecture

- Input layer: 2 neurons
- Hidden layer: 2 neurons (with ReLU or Sigmoid activation)
- Output layer: 1 neuron (with Sigmoid)
- Target value(Output)=1
- Compute Loss or MSE

Problem-contd

Input (X):

$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

Weights & Biases:

Hidden layer:

$$W_1 = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.4 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Output layer:

$$W_2 = \begin{bmatrix} 0.6 & 0.9 \end{bmatrix}, \quad b_2 = 0.0$$

Activation Functions:

- Hidden: **ReLU** or **Sigmoid**
- Output: **Sigmoid**

🔁 Step 1: Forward Pass

1. Hidden layer pre-activation:

$$Z_1 = W_1 \cdot X + b_1 = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.4 \end{bmatrix} \cdot \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.1 \cdot 1.0 + 0.3 \cdot 0.5 \\ 0.2 \cdot 1.0 + 0.4 \cdot 0.5 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.4 \end{bmatrix}$$

2. Hidden layer activation:

$$A_1 = \sigma(Z_1) = \begin{bmatrix} \sigma(0.25) \\ \sigma(0.4) \end{bmatrix} = \begin{bmatrix} 0.562 \\ 0.598 \end{bmatrix}$$

3. Output layer pre-activation:

$$Z_2 = W_2 \cdot A_1 + b_2 = \begin{bmatrix} 0.6 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.562 \\ 0.598 \end{bmatrix} = 0.6 \cdot 0.562 + 0.9 \cdot 0.598 = 0.337 + 0.538 = 0.875$$

4. Output activation:

$$\hat{y} = \sigma(Z_2) = \sigma(0.875) \approx 0.706$$

🎯 Step 2: Compute Loss

Assume target $y = 1$, and we use **Mean Squared Error (MSE)**:

$$L = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - 0.706)^2 \approx 0.043$$

🔁 Step 3: Backward Pass (Backpropagation)

A. Derivative of Loss w.r.t. output:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.706 - 1 = -0.294$$

B. Derivative w.r.t. Z_2:

$$\frac{\partial \hat{y}}{\partial Z_2} = \hat{y}(1 - \hat{y}) = 0.706(1 - 0.706) \approx 0.207 \Rightarrow \frac{\partial L}{\partial Z_2} = -0.294 \cdot 0.207 = -0.0608$$

C. Gradients for W2 and b2:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial Z_2} \cdot A_1^T = -0.0608 \cdot \begin{bmatrix} 0.562 & 0.598 \end{bmatrix} \approx \begin{bmatrix} -0.0342 & -0.0363 \end{bmatrix}$$

$$\frac{\partial L}{\partial b_2} = \downarrow 0.0608$$

C. Gradients for W2 and b2:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial Z_2} \cdot A_1^T = -0.0608 \cdot \begin{bmatrix} 0.562 & 0.598 \end{bmatrix} \approx \begin{bmatrix} -0.0342 & -0.0363 \end{bmatrix}$$

$$\frac{\partial L}{\partial b_2} = -0.0608$$

D. Gradients for W1 using chain rule:

Backprop into hidden layer:

$$\delta_1 = \frac{\partial L}{\partial Z_2} \cdot W_2 \cdot \sigma'(Z_1)$$

$$\sigma'(Z_1) = A_1 \cdot (1 - A_1) = [0.562(1 - 0.562), 0.598(1 - 0.598)] \approx [0.246, 0.241]$$

$$\delta_1 = -0.0608 \cdot [0.6, 0.9] \cdot [0.246, 0.241] = [-0.009, -0.0132]$$

Final gradients for W_1:

$$\frac{\partial L}{\partial W_1} = \delta_1 \cdot X^T = \begin{bmatrix} -0.009 \\ -0.0132 \end{bmatrix} \cdot \begin{bmatrix} \downarrow & 0.5 \end{bmatrix} = \begin{bmatrix} -0.009 & -0.0045 \\ -0.0132 & -0.0066 \end{bmatrix}$$

Step 4: Gradient Descent (Update Rule)

With learning rate $\eta = 0.1$:

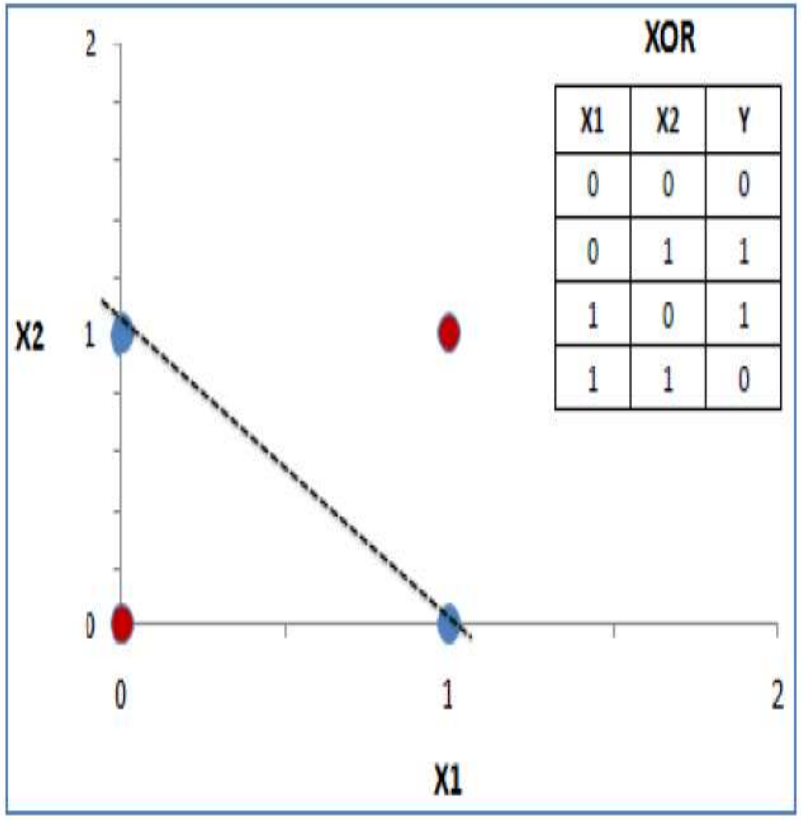
$$W_1 = W_1 - \eta \cdot \frac{\partial L}{\partial W_1}$$

$$W_2 = W_2 - \eta \cdot \frac{\partial L}{\partial W_2}$$

And same for biases.

Learning XOR Gate

An XOR (exclusive OR gate) is a digital logic gate that gives a true output only when both its inputs differ from each other. The truth table for an XOR gate is shown below:

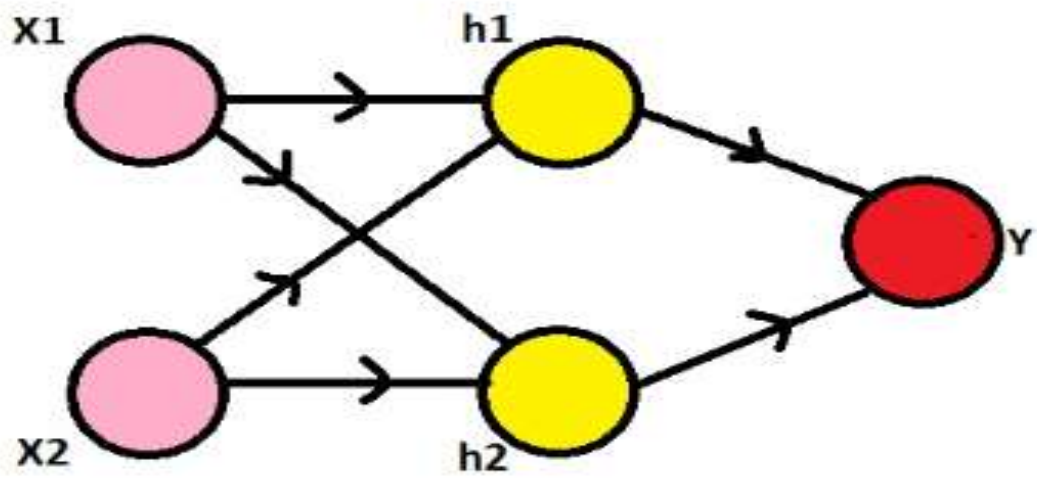


The goal of the neural network is to classify the input patterns according to the above truth table. If the input patterns are plotted according to their outputs, it is seen that these points are not linearly separable. A single neuron is not able to solve an XOR problem

XOR problem

- To solve this problem, we add an extra layer to perceptron, i.e., we create a **Multi Layered Perceptron** (or **MLP**).
- We call this extra layer as the **Hidden layer**. To build a perceptron, we first need to understand that the XOR gate can be written as a combination of AND gates, NOT gates and OR gates in the following way:
- A *XOR* B =(AB'+A'B)

XOR problem solution



we need to observe that our inputs are 0s and 1s.

•To make it a XOR gate, we will make the h2 node to perform the (x1x2)' operation,

•h1 node to perform (x1 + x2) operation and the y node to perform (h1 **AND** h2) operation.

•Also, we'll use the sigmoid function as our activation function σ, i.e., σ(x) = 1/(1+e^(-x)) and the threshold for classification would be 0.5, i.e., any x with σ(x)>0.5 will be classified as 1 and others will be classified as 0.

XOR problem solution

we can also represent XOR as

- a XOR b=(a + b)(ab)'

  = (a+b)(a'+b')=ab'+a'b

- In this case, 'h1' node perform (a+b) and 'h2' node perform (ab)'
- 'y' node perform h1 AND h2

Solving XOR problem Example

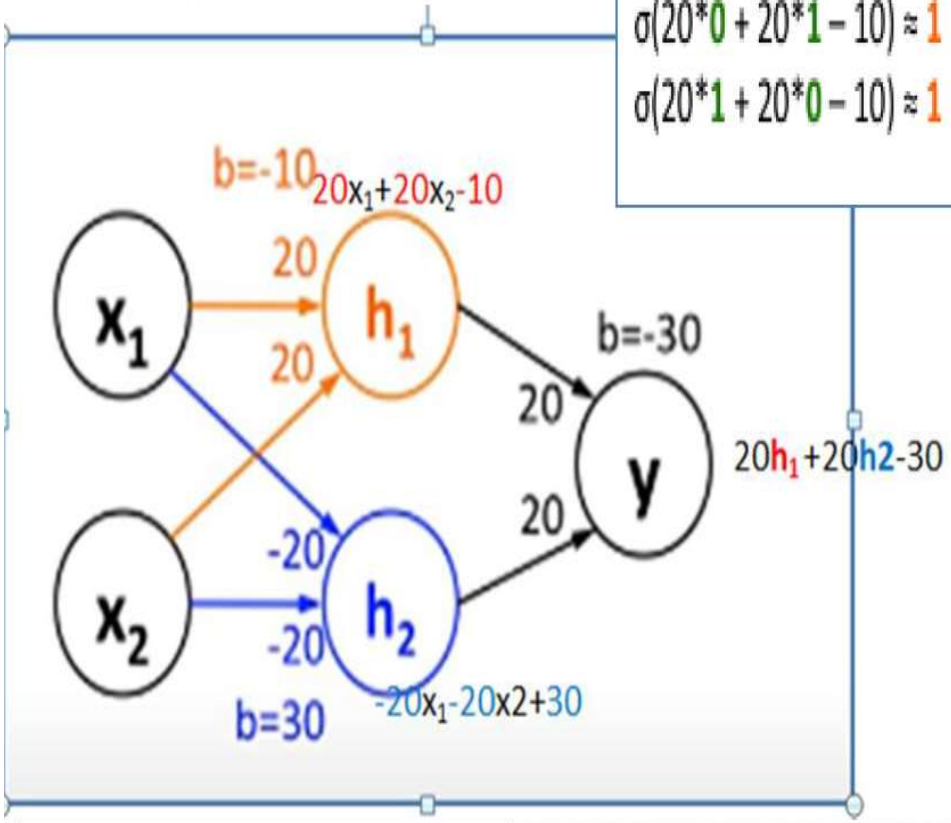
Consider the following network with 2 inputs and 1 hidden layer.

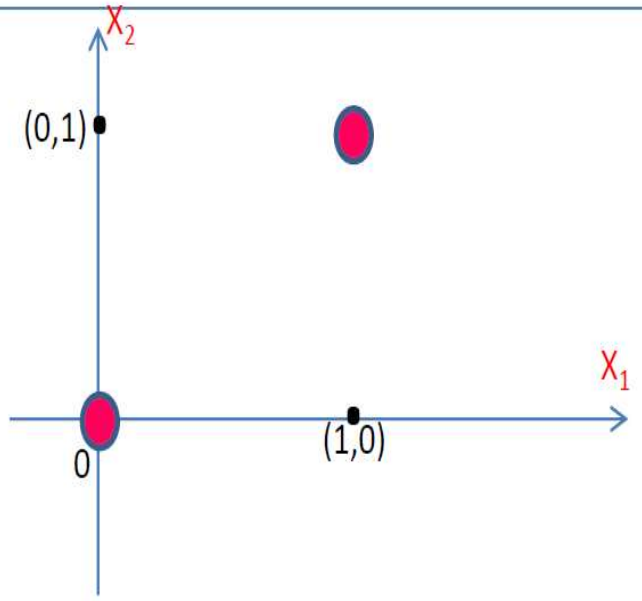
$\sigma(20*0+20*0-10) \approx 0$ $\sigma(-20*0-20*0+30) \approx 1$ $\sigma(20*0+20*1-30) \approx 0$

$\sigma(20*1+20*1-10) \approx 1$ $\sigma(-20*1-20*1+30) \approx 0$ $\sigma(20*1+20*0-30) \approx 0$

$\sigma(20*0+20*1-10) \approx 1$ $\sigma(-20*0-20*1+30) \approx 1$ $\sigma(20*1+20*1-30) \approx 1$

$\sigma(20*1+20*0-10) \approx 1$ $\sigma(-20*1-20*0+30) \approx 1$ $\sigma(20*1+20*1-30) \approx 1$





In this,h1=x1+x2,h2=(x1x2)' and y=h1.h2

Solving XOR problem Example

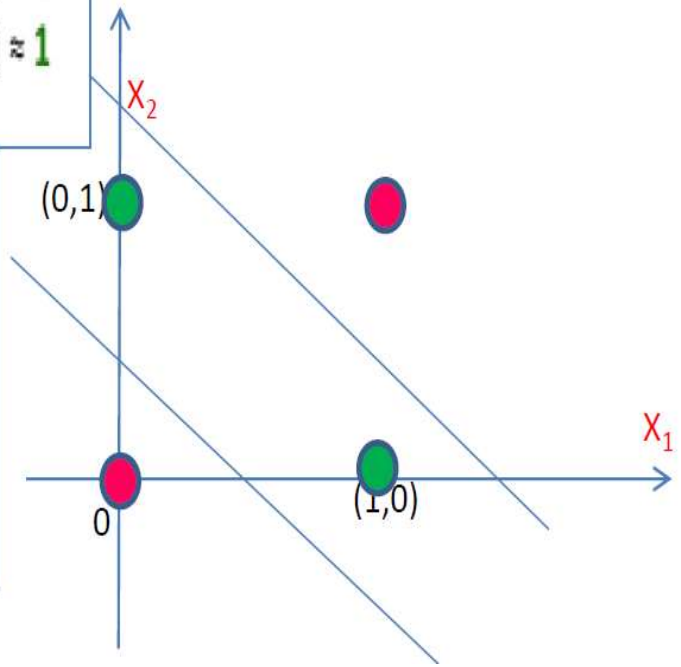
$\sigma(20*0+20*0-10) \approx 0$ $\sigma(-20*0-20*0+30) \approx 1$ $\sigma(20*0+20*1-30) \approx 0$

$\sigma(20*1+20*1-10) \approx 1$ $\sigma(-20*1-20*1+30) \approx 0$ $\sigma(20*1+20*0-30) \approx 0$

$\sigma(20*0+20*1-10) \approx 1$ $\sigma(-20*0-20*1+30) \approx 1$ $\sigma(20*1+20*1-30) \approx 1$

$\sigma(20*1+20*0-10) \approx 1$ $\sigma(-20*1-20*0+30) \approx 1$ $\sigma(20*1+20*1-30) \approx 1$

MLP solves the XOR problem efficiently by visualizing the data points in multi-dimensions and thus constructing an n-variable equation to fit in the output values(hyperplane).



Types of gradient Descent:

- **Batch Gradient Descent:** This is a type of gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive and not preferred. Instead, we prefer to use stochastic gradient descent or mini-batch gradient descent.
- **Stochastic Gradient Descent:** This is a type of gradient descent which processes 1 training example per iteration. Hence, the parameters are being updated even after one iteration in which only a single example has been processed. Hence this is quite faster than batch gradient descent. But again, when the number of training examples is large, even then it processes only one example which can be additional overhead for the system as the number of iterations will be quite large.
- Mini Batch gradient descent: This is a type of gradient descent which works faster than both batch gradient descent and stochastic gradient descent. Here b examples where b<m are processed per iteration. So even if the number of training examples is large, it is processed in batches of b training examples in one go. Thus, it works for larger training examples and that too with lesser number of iterations.

Types of gradient Descent:

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini batch gradient descent
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights	Use **a batch of** (randomly picked) samples for a forward pass and then adjust weights